

Stress-testing the `tanglepack` numerics layer

What the manifold-growth engine can and cannot do, with evidence

`tanglepack` numerics stress campaign

2026-10-07

This report presents a 59.8-minute stress campaign of 349 isolated runs (ten experiments, E1–E10), followed by a 24-minute deep-blasting study of 53 runs (E11), 402 runs in all, against `tanglepack.numerics`, the layer that grows the stable and unstable manifolds of saddle orbits of area-preserving maps, detects their crossings and builds bridges. Each section follows the same pattern: a claim, the figure or table that supports it, an interpretation, and a caveat. Negative results count as findings. Every measured number in the text is generated from the raw result files (`numbers.tex`, `numbers_extra.tex`); none is typed by hand.

Branch `numerics-stress`, library commit `e963893`. The library itself was not modified.

Contents

1	Executive summary	1
2	Setup and methods	4
3	Performance and scaling (E3)	5
4	GPU acceleration (E1, E2)	7
4.1	Raw throughput	7
4.2	End to end	7
4.3	Why: Amdahl's law	7
5	Accuracy versus cost (E4)	9
6	Physical invariants (E5)	10
7	Correctness of crossing detection (E9)	12
8	Robustness across parameters (E6)	13
9	Limits and deep exploration (E7)	13
9.1	Escape to infinity	15
9.2	Depth	15
9.3	Blasts	16
9.4	Near tangency: artefact crossings	17
10	Deep blasting: how the topology evolves (E11)	17
10.1	Design of the sweep	18
10.2	How the topology evolves	18
10.3	Cost of blasting: the practical depth limit	19
10.4	Dependence on parameters, and determinism	20
10.5	Provisional symbolic results as the tangle grows	21
10.6	Genuine errors	24
11	Fixed-point solver (E8)	25
12	Reproducibility (E10)	26
13	Failure taxonomy	28
14	Recommendations	28
15	Threats to validity	30
A	Tables	32
B	Reproduction	33
C	Configuration and outcome counts	34

1 Executive summary

Six headline findings.

1. **The GPU does not help, and cannot help in the current design.** The end-to-end speed-up is $0.999 \times$ (k28), $0.975 \times$ (p3) and $0.958 \times$ (k10). The map takes at most 0.84% of growth time, so Amdahl’s law caps the gain at $1.008 \times$ (Section 4).
2. **Cost is linear in the point count, but the point count is not set by the dynamics.** Step time scales as $N^{1.06}$ and memory as 1 058 B/point. However, once the unstable arm escapes to infinity a single step multiplies the points by up to 61.8, against $\lambda_u = 7.15$ (Section 3).
3. **area_cutoff is a well-behaved accuracy knob.** The invariance error falls as $\text{cutoff}^{0.70}$ and the point count grows as $\text{cutoff}^{-0.332}$ (k28), close to the $2/3$ and $-1/3$ that chord geometry predicts. At 10^{-7} the median invariance error on k28 is 1.8×10^{-6} for 0.107s of pipeline time. The library default, 10^{-4} , is too coarse for the period-3 case: it invents and drops crossings (Section 5).
4. **Transverse crossings are found exactly, and the physical invariants hold.** An independent brute force matches all 154/154 crossings with 0 missed and 0 extra. Lobe areas are preserved to 5.0×10^{-3} , and 0 cdist decreases occur in 77 328 steps (Sections 6 and 7).
5. **Two failures are silent.** (a) Near the saddle-centre bifurcation, at fixed $k + 1 = 10^{-4}$ and a fixed manifold length, the number of registered crossings grows from 25 to 499 as the resolution is refined, with 0 warnings. These are artefacts of the polygons, not dynamics. (b) The fixed-point solver hands back lower-period orbits: 1 424 of them were accepted as saddles of the requested period (Sections 9 and 11).
6. **Deep blasting keeps uncovering structure, no crossing or cdist law is broken, and the topology layer has four genuine errors to triage.** Over 886 blasts in 53 configurations there were 0 same-stability crossings or cdist decreases; the lobe-area check is open in the two-iteration cells. The canonical words changed in 490 snapshots (k28: $3 \rightarrow 57$ classes), and memory limits the large zones to 14–15 blasts at $\times 1.65$ points per blast (k28). The topology raised in 137 of 939 snapshots (23 configurations), with 4 distinct errors, the most frequent being a zero-width iterated element (Section 10).

Figure 1 puts one panel per theme side by side, and Table 1 grades each capability with a pointer to its evidence. Section 13 catalogues the failures, and Section 14 expands the recommendations below.

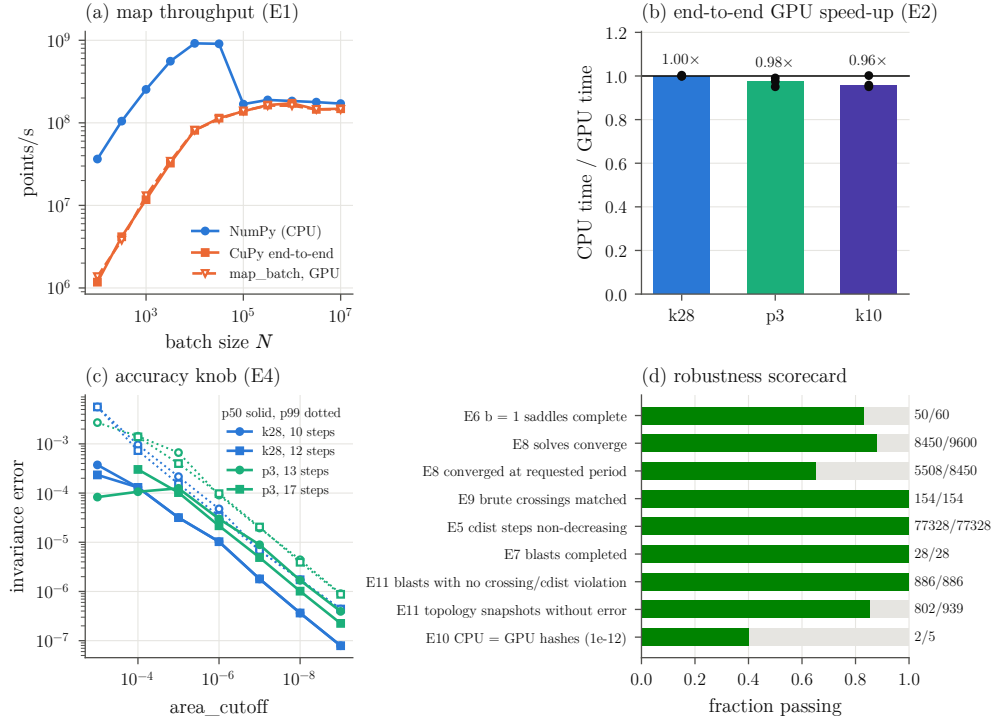


Figure 1: Campaign overview. (a) Map throughput: NumPy beats the end-to-end CuPy path at every batch size, and the `map_batch` GPU path only catches up at the largest batches (Section 4). (b) End-to-end speed-up per case: every bar is at or below 1. (c) Invariance error falls steadily as `area_cutoff` is refined; solid is the median, dotted the 99th percentile. (d) Pass fractions of the main checks. The short bars are the solver’s requested-period rate and the CPU/GPU hash agreement; both are explained in Sections 11 and 12.

Table 1: Scorecard. Status: **works** = does what it should over the tested range; **limited** = works, with a documented restriction or with genuine errors (iii) in a stated corner; **fails** = does not deliver. Topology change (i) and provisional flags (ii) never count against a capability (Section 2).

capability	status	evidence
Manifold growth, stop-condition driven	works	escape case stops at 478 points (Section 9.1)
Manifold growth, fixed step count	fails	killed at the 20 GB cap after 9 steps (Section 9.1)
Cost scaling	works	time $\propto N^{1.06}$, 1 058 B/point (Section 3)
Accuracy control (<code>area_cutoff</code>)	works	error $\propto \text{cutoff}^{0.70}$ (Section 5)
Transverse crossing detection	works	154/154 matched, 0 extra (Section 7)
Crossings near tangency	fails	count tracks resolution, 0 warnings (Section 9.4)
Self-crossing detection	limited	4 $u \times u$ crossings exist, none reported (Section 7)
Area preservation, invariants	works	lobe $ A'/A - 1 \leq 5.0 \times 10^{-3}$ (Section 6)
Arrangement <code>image_of</code>	–	47 images found, 39 missing; open question, not graded (Section 6)
Blasting (numerics)	works	886 blasts in 53 configs, 0 crossing/cdist law violations (area probe open in the two-iteration cells), deterministic to the hash; crossings 4→7 792 and classes 3→57 (k28) (Sections 9.3 and 10)
Blast depth, large zones	limited	points $\times 1.65$ per blast; at cutoff 10^{-7} the memory budget ends k28 at 15 blasts (9.0 GB), at coarser cutoffs the time budget; p3 runs all 20 in 0.42 GB (Section 10.3)
Topology under deep blasting	limited	137/939 snapshots raised (4 genuine errors, 23/53 configs); the other snapshots completed, some with provisional symbolic results (open, needs more growth) (Section 10.6)
Deep growth (p3)	limited	works through 17 steps, the tangle growing with each step; OOM (genuine error) at 18 (Section 9.2)
Inversion saddle	limited	topology <code>AssertionError</code> at 4 and 5 steps and OOM at 7 and 8 (genuine errors); at 6 steps it completes with a provisional symbolic result (Section 9.2)
Parameter robustness ($b = 1$ saddle)	limited	83.3% complete; 26 trivial tangles (Section 8)
Fixed-point solver	limited	converges 88%, but accepts lower periods (Section 11)
GPU acceleration	fails	$0.958 \times - 0.999 \times$ end to end (Section 4)
Determinism	works	CPU–CPU and GPU–GPU bit-identical (Section 12)

What I would do next, in priority order.

1. **Guard the crossing registration near tangency.** Warn when a crossing’s $|\sin \theta|$ is not large compared to the local invariance error, and add a cheap $u \times u$ self-crossing check as a resolution diagnostic. Both failures are currently silent.
2. **Make the solver check the minimal period** before `construct_fixed_point` accepts an orbit.
3. **Put a point/memory budget into growth.** Predict the next step’s points from the last growth factor, as the E3 harness already does, and make stop-condition growth the documented default.
4. **Raise the default `area_cutoff` to 10^{-7} ,** which is what the k28, p3 and nested test cases already use.
5. **Triage the four E11 topology errors** (Section 10.6), starting with the zero-width iterated element, which hits every k28 and k32 cell at cutoff 10^{-7} .
6. **Leave the GPU off.** Revisit it only if refinement and the linked list move to arrays that live

on the device.

2 Setup and methods

The map and the pipeline. All runs use the Hénon family (parameters k , b ; $b = 1$ is area preserving) defined in the module `tanglepack.examples.henon`, via the frozen recipes of `tests/cases.py`. These recipes are: `k10`, the $k = 10$ binary horseshoe at the default cut-off 10^{-4} ; `k28`, $k = 2.8$ at 10^{-7} , optionally blasted; `p3`, the period-3 orbit of the nested map; `nested`, period 1 plus period 3; and `inversion`, the other saddle of $k = 10$, with both eigenvalues negative. The pipeline under test is `TangleWorkbench`: find the fixed point, then initialise the fundamental segments, grow the manifolds (`grow_n_times`, `grow_until_*`), and finally run `compute_intersections`, `trim`, `create_bridges` and the iterate table. Where an experiment touches topology (E5, E7), it is only as a downstream consumer of the numerics.

Topology change, provisional results and errors. Blasting a zone or growing the manifolds further is *meant* to change the topology: new crossings are registered, new bridges, holes and bridge classes appear, elements split, and words, letters and transition matrices change. This is new information about the tangle, and the report never counts it as a failure. The report keeps three things apart. (i) *Topology change*: the expected outcome of more growth or more blasts. (ii) *Provisional symbolic results*: the library’s own flags (`is_reliable=False`, ambiguous walks, unresolved classes, virtual classes `new1`, ...). They mean the symbolic computation at that stage is incomplete: some class images are not registered yet, or a dual-graph walk cannot yet be completed, and they call for more growth or blasting. They are open questions, not numerical errors. (iii) *Errors*: exceptions, invariant `AssertionErrors`, memory or time exhaustion, and numerics that violate a physical law (a $u \times u$ or $s \times s$ crossing, area not preserved, `cdist` decreasing). Topology change (i) and provisional flags (ii) never count against a capability; only (iii) does. The scorecard grade also reflects whether the capability delivers (e.g. the GPU). A crossing count that depends on `area_cutoff` at *fixed* depth is a resolution error (Sections 5 and 9.4); a count that grows with depth or blasts is (i).

Harness. `stress/numerics/harness.py` runs every configuration in a fresh subprocess with a hard timeout. A `psutil` watchdog samples the worker’s resident set every 50 ms and kills the worker above an RSS cap. `RLIMIT_AS` is not used because CUDA reserves huge virtual ranges. Each configuration writes one JSON line with its parameters, per-stage wall times (with a device synchronise before the clock on the GPU), per-step checkpoints, metrics, captured `logging` warnings by template, peak RSS and git SHA. The outcome is one of `ok`, `exception`, `invariant_assert`, `oom`, `timeout` or `crash`. *Failures are data*: a sweep never stops on one, and the per-step checkpoints keep everything up to the failure. The per-experiment caps are in Table 12. Timing experiments (E1–E4, E10) ran serially with the GPU never shared; the robustness sweeps (E5–E9) ran in worker pools.

Hardware and software. One host (`pop-os`): Intel Core i9-14900K (24 cores, 8P + 16E, 32 threads, 32 MB L2, 36 MB L3), 125 GB RAM; NVIDIA GeForce RTX 5060 Ti, 16 GB, driver 580.173.02; Linux 7.1.1. Python 3.10.12, NumPy 2.2.5, SciPy 1.15.1, CuPy 14.1.1, `rtree` 1.4.0. Every record carries library commit `e963893`. The full campaign took 59.8 min of wall-clock, after a 1.2-minute `--quick` pilot; E11 ran on its own afterwards in 24 min.

Experiments.

E1 Raw map throughput

NumPy against CuPy (end to end, compute only, transfer only) and the real `map_batch` path, $N = 10^2$ – 10^7 .

E2 End-to-end GPU

k28, p3 and k10, CPU against GPU, three repeats; also a cProfile breakdown, a sweep of `min_batch_points` and a lockstep CPU/GPU comparison.

E3 Scaling

Growth step by step to $\sim 2 \times 10^6$ points for $k \in \{2.8, 4, 6, 10\}$, and downstream stages as a function of the segment count.

E4 Accuracy against cost

`area_cutoff` 10^{-3} – 10^{-9} on k28 and p3 at two depths, against the finest run.

E5 Invariants

Fixed point, manifold invariance, iterate links, lobe areas, cdist monotonicity and `image_of`, on every frozen case.

E6 Robustness

One fixed recipe over $k \in [0.3, 15]$ for $b = 1$ (both period-1 points), $b = -1$ and dissipative b .

E7 Limits and deep exploration

Escape to infinity and near-tangency at $k \rightarrow -1$, where the numerics are expected to struggle; 0–10 blasts and deep growth, where the tangle is expected to reveal new structure.

E8 Solver

200 random seeds $\times$ 2 guess styles, periods 1–6, four maps.

E9 Crossing correctness

The library’s crossings against an exact $O(NM)$ brute force.

E10 Determinism

Five fresh-process repeats on the CPU and five on the GPU.

E11 Deep blasting

Five zones blasted up to 20 times, with the full topology stack snapshotted after every blast, over `min_separation`, blasting `area_cutoff`, iterations per blast and repeats (Section 10).

3 Performance and scaling (E3)

Claim. Growth, crossing detection and bridge construction all cost essentially linear time in the number of points, and memory costs about 1058 B per point. The binding constraint is not the algorithm but the number of points the escaping unstable arm generates.

Evidence. Figure 2(a) shows the time of one growth step against the points it produces, for four values of k and two cutoffs. Above 10^3 points the fitted slopes run from 1.05 to 1.08 (median 1.06). The downstream stages in Figure 2(b) scale with the segment count with slopes 1.07 (`compute_intersections`) and 0.958 (trim and bridges). The iterate table is flat (0.0391) because it depends on the crossings, not on the points. Resident memory (Figure 3) grows by 1058 B per point (median over runs; maximum 1075).

Figure 4 explains why the point count is the real problem. For the first steps the points grow roughly like $|\lambda_u|^n$. Then, at step 9 ($k = 10$) to 11 ($k = 2.8$), the largest coordinate passes 100 and the growth factor departs from λ_u . On its last completed step the $k = 6$ manifold grew by a factor 61.8 ($\lambda_u = 7.15$), $k = 10$ by 38.6 ($\lambda_u = 8.52$) and $k = 2.8$ by 24.0 ($\lambda_u = 5.72$). Meanwhile the largest coordinate reached 3.5×10^7 and 6.6×10^6 .

Interpretation. The refinement places points to keep a fixed chord area, so the point count follows the arclength and curvature of the image, not the eigenvalue. Once the tip of the arm is at $|x| \sim 10^3$, one step of a quadratic map sends it to $\sim 10^6$, and the refinement faithfully resolves an arc that is irrelevant to the tangle. “More steps” therefore mostly buys points far outside the

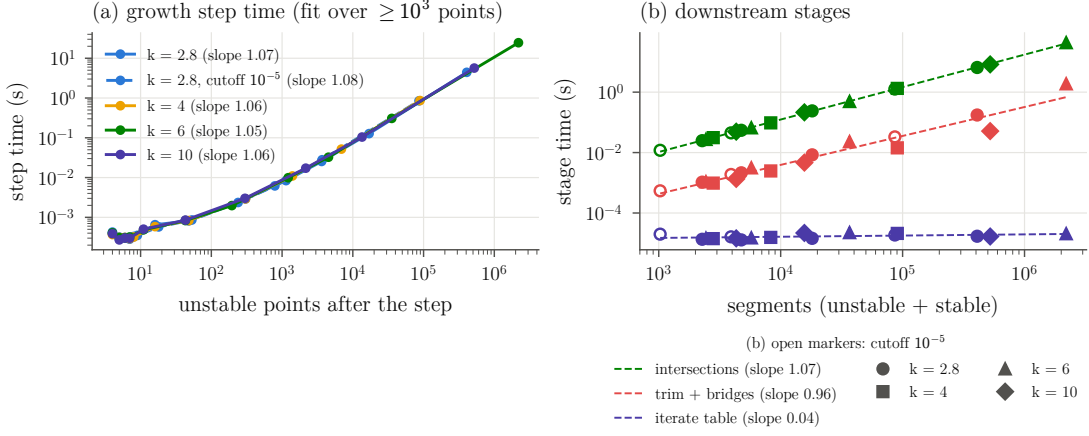


Figure 2: E3 cost. (a) Growth-step time against unstable points: all k and both cutoffs collapse onto one line with slope near 1 (legend), so cost depends on the point count alone. (b) Downstream stages against segments: intersections and trim/bridges are linear; the iterate table does not depend on the point count.

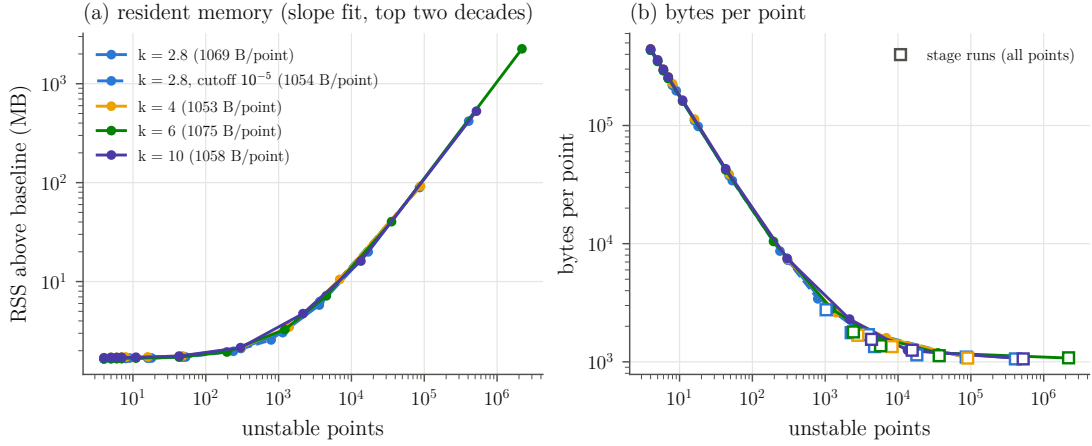


Figure 3: E3 memory. (a) RSS above the interpreter baseline: linear over the top two decades, with nearly the same slope for every k (legend). (b) Bytes per point: fixed interpreter overhead dominates small manifolds; the curve flattens onto the per-point slope for large ones.

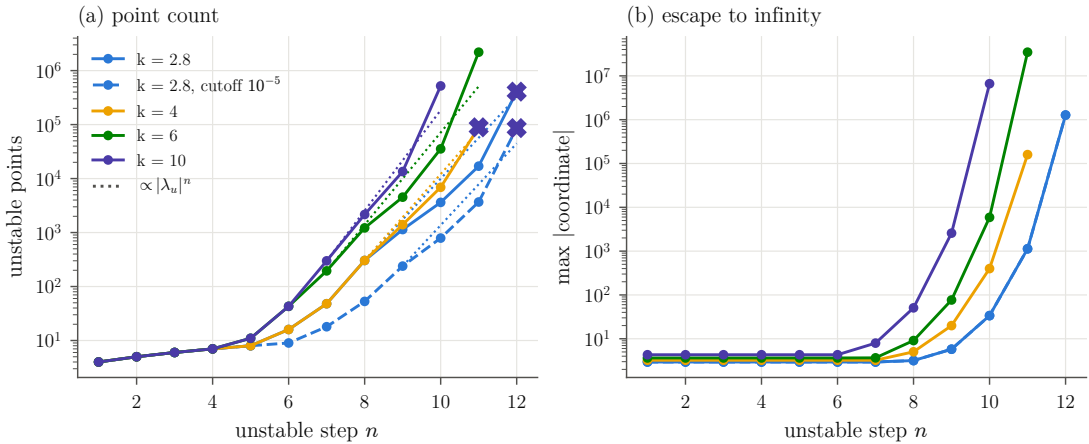


Figure 4: E3 growth. (a) Unstable points per step against the $|\lambda_u|^n$ guide (dotted). The curves follow it for a few steps and then shoot above it; crosses mark runs killed at the 24GB cap. (b) The largest coordinate: the take-off in (a) coincides with the arm leaving for infinity.

tangle. 3 growth runs were killed at the 24 GB cap. At that moment they held only 86 168–407 709 points after their last completed step, but they had spent 190–208 s inside the step that was killed.

Caveat. The 1058 B/point slope gives a ceiling of roughly 2.4×10^7 points in a 24 GB budget. This is an *estimate*: it extrapolates the steady-state slope, and a step’s transient working set comes on top of it. The exponents are fitted over ≤ 3 decades on one machine.

4 GPU acceleration (E1, E2)

Claim. Enabling the GPU (`enable_gpu`) does not speed up growth. Today it is neutral at best and slower at worst, and it cannot pay off while the map is a negligible share of the work.

4.1 Raw throughput

Evidence. Figure 5(a): NumPy peaks at 9.2×10^8 points/s near $N = 10^4$, then drops by $5.4\times$ to 1.7×10^8 points/s. This is consistent with a cache effect: the working set leaves the per-core L2. CuPy on data already on the device reaches 1.6×10^9 points/s, but the end-to-end path (host $\rightarrow$ device $\rightarrow$ host, which is what `gpu.py` does) peaks at 1.7×10^8 . At $N = 10^7$, transfer is 88% of the GPU time (Figure 5(b)). The raw crossover: none up to $N = 10^7$ (end-to-end CuPy reaches $0.87\times$ of NumPy at the largest N). Through `map_batch`, whose CPU path pays for transposes and copies, the GPU breaks even somewhere between 6.2×10^5 (interpolated from the sweep) and 4.1×10^6 (bisection). At 10^7 it is ahead by only $1.11\times$.

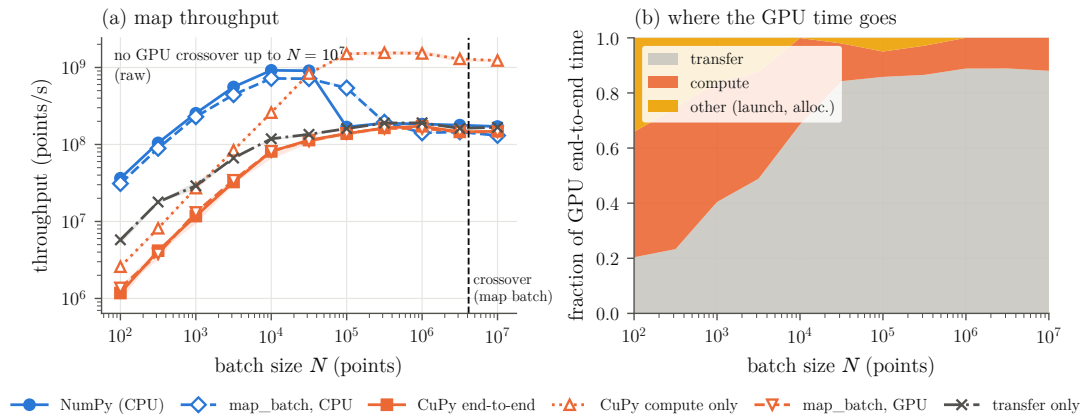


Figure 5: E1. (a) Throughput against batch size. Device-only compute (dotted triangles) is fastest, but every path that includes the transfer sits on the transfer-only line (grey crosses). The dashed line marks the bisected `map_batch` crossover; read it as the upper edge of a band, not a sharp threshold. (b) Share of GPU end-to-end time: transfer dominates from mid-size batches on.

4.2 End to end

Evidence. Figure 6 and Table 2: the summed pipeline time on the GPU is $0.999\times$ (k28), $0.975\times$ (p3) and $0.958\times$ (k10) of the CPU speed (median of three repeats; best repeats $1.004\times$, $0.991\times$, $1.002\times$). The best single growth step, k28 at 4.3×10^6 points, gains $1.007\times$. On the E10 case the GPU is clearly slower: $0.85\times$ (Section 12).

4.3 Why: Amdahl’s law

Evidence. Figure 7: in the cProfile of unstable growth the map is 0.0171% (k28), 0.505% (p3) and 0.84% (k10) of the CPU time. The bounds are $1.000\times$, $1.005\times$ and $1.008\times$. Refinement plus linked-list work is 83% (k28), 75% (p3) and 76% (k10). In the k28 profile (83s), `_refine_layer`

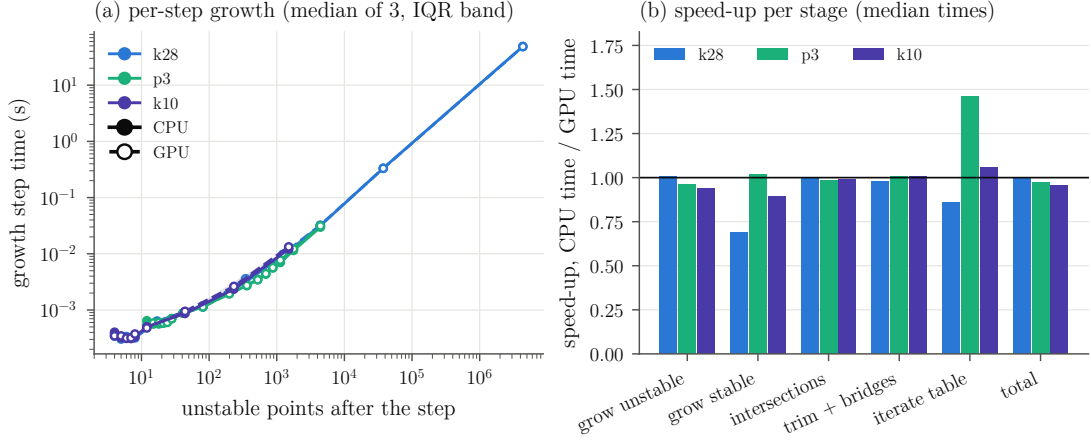


Figure 6: E2. (a) Per-step growth time; CPU (filled) and GPU (open) markers lie on top of each other up to 4.3×10^6 points. (b) Per-stage speed-up. Everything sits within a few percent of 1 except tiny stages, where sub-millisecond noise dominates (p3 iterate table, k28 stable growth).

Table 2: E2: median CPU time / median GPU time per stage, the CPU map share f_{map} (cProfile) and the Amdahl bound $1/(1 - f_{\text{map}})$.

case	grow unstable	grow stable	intersections	trim + bridges	iterate table	total	f_{map}	Amdahl
k28	$1.007\times$	$0.689\times$	$1.001\times$	$0.978\times$	$0.858\times$	$1.001\times$	0.0171%	$1.000\times$
p3	$0.962\times$	$1.017\times$	$0.985\times$	$1.006\times$	$1.462\times$	$0.970\times$	0.505%	$1.005\times$
k10	$0.939\times$	$0.895\times$	$0.991\times$	$1.006\times$	$1.060\times$	$0.958\times$	0.84%	$1.008\times$

alone takes 44% of the self time. `Point.__init__` runs 8.7×10^6 times (18% cumulative), each paired with a `numpy.array` call (8.7×10^6). The `min_batch_points` sweep (Figure 8) is flat: the best setting (1 024) is $1.011\times$ faster than “never”, which is inside the run-to-run noise of Section 12.

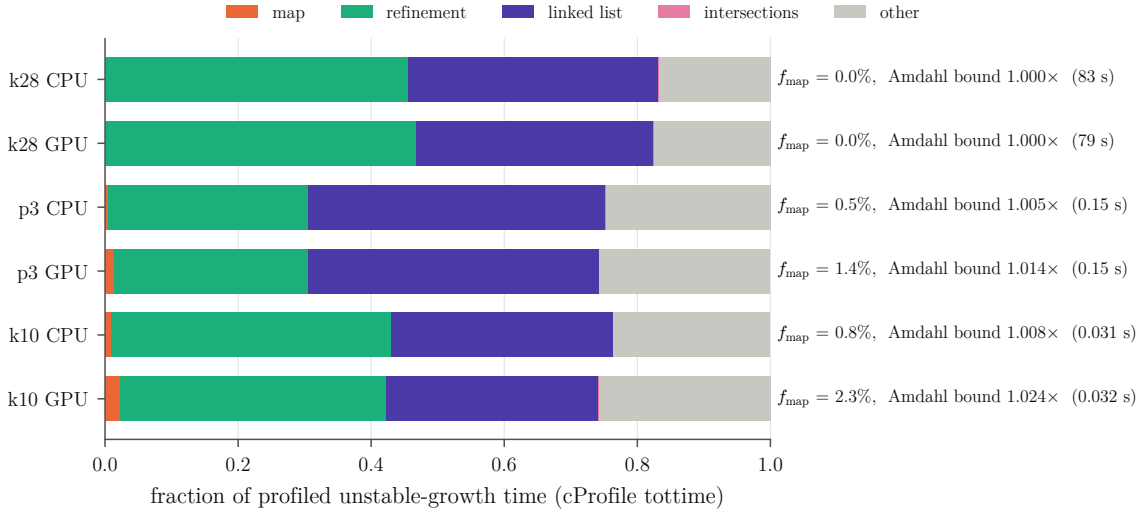


Figure 7: E2 cProfile breakdown of unstable growth (self time by category). The map (orange) is invisible at this scale; refinement and the linked list (Python objects, pointer walks) take 75%–83% of it. The Amdahl bound on the right is the most any map accelerator could give.

Interpretation. The GPU path accelerates the only part of growth that was already fast. Growth is a Python object graph: one `Point` per vertex, two doubly linked lists, and refinement that walks them layer by layer. For the GPU to pay, refinement and the linked list must become arrays (coordinates, cdists, iterate indices), with the whole growth loop resident on the device. Only then would f_{map} become the GPU-friendly bulk of the work. Short of that, the measured ceiling is

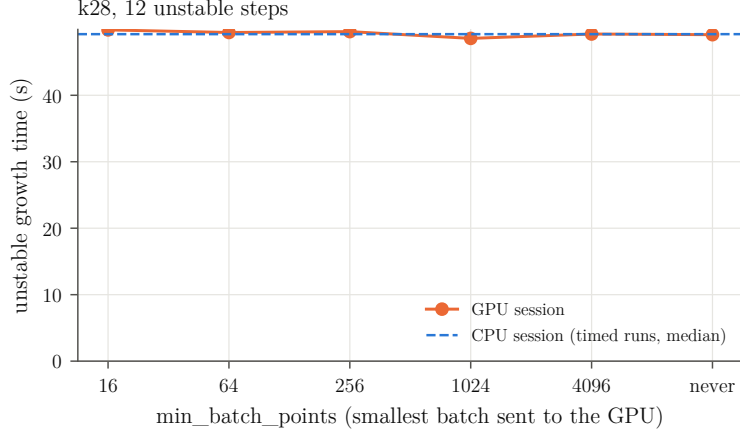


Figure 8: E2 `min_batch_points` sweep on k28 (12 unstable steps). The GPU session is flat across the threshold and matches the CPU median (dashed): the threshold has nothing to tune.

1.008 $\times$.

Caveat. cProfile inflates the per-call overhead of Python functions relative to NumPy kernels, so the true f_{map} is probably a little larger than measured. Even ten times the measured share would keep the bound close to 1. Three repeats per configuration resolve differences of a few percent, not less.

5 Accuracy versus cost (E4)

Claim. `area_cutoff` trades accuracy for points along a predictable power law. The default 10^{-4} is adequate for the period-1 cases but not for p3.

Evidence. Figure 9(a) plots invariance error against pipeline time. The invariance error is the distance from the image of a point inside a polyline segment to the polyline of the image branch; the probe samples segment interiors because a vertex’s image is, by construction, often a vertex. On k28 (10 steps), going from 10^{-4} to 10^{-7} moves the median error from 1.3×10^{-4} to 1.8×10^{-6} and the 99th percentile from 9.6×10^{-4} to 6.9×10^{-6} . The cost goes from 0.0148s to 0.107s and from 450 to 4757 points. Over cutoffs 10^{-6} – 10^{-9} the median error has slopes 0.70 and 0.70 (k28, 10 and 12 steps), and 0.64 and 0.66 (p3, 13 and 17 steps). The point count has slope -0.332 (k28) and -0.272 (p3) (Figure 10(b)). Crossing positions converge to the finest run with slopes 0.73, 0.70, 0.66 and 0.58 (Figure 10(a)). The full grid is in Table 10.

Interpretation. The exponents are what chord geometry predicts. A chord of length ℓ on a curve of curvature κ deviates by $h \approx \kappa \ell^2 / 8$ and encloses an area $\propto \kappa \ell^3$. A fixed area cutoff A therefore gives $\ell \propto A^{1/3}$, so points $\propto A^{-1/3}$ and a geometric error $h \propto A^{2/3}$. The crossing *count* is what matters for topology. For p3 at 13 steps the default 10^{-4} registers 37 crossings and 10^{-3} registers 14, while every cutoff from 10^{-5} on gives 27. At 17 steps, 10^{-4} gives 249 and 10^{-5} gives 214, against 203 from 10^{-7} on. A coarse polygon both misses true crossings and creates spurious ones. A count that depends on the cutoff at fixed depth is a resolution error; a count that grows with depth or blasts is the tangle growing. 10^{-7} is the cheapest setting that is converged in every case tested.

Caveat. The reference is the finest run (10^{-9}), not an exact manifold, so the convergence plots measure self-consistency. The invariance error is a reference-free proxy, but it measures one-step consistency, not distance to the true manifold. p3 at 17 steps and 10^{-3} ran out of memory (16 GB cap): a coarse polygon of the escaping arm is not cheaper.

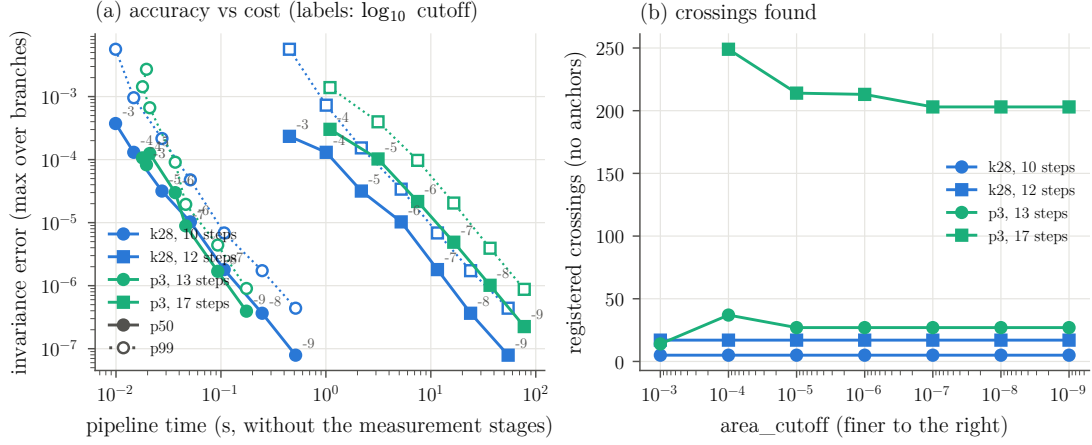


Figure 9: E4. (a) Accuracy against cost; labels are $\log_{10}$ of the cutoff. Every case traces a straight Pareto front, so there is no knee and no cheap win. (b) Registered crossings per cutoff. k28 is stable at every cutoff; p3 needs $\leq 10^{-5}$ (13 steps) or $\leq 10^{-7}$ (17 steps) before the count settles. Counts are compared at fixed depth; more steps register more crossings (k28: 5 at 10 steps, 17 at 12), which is new tangle, not error.

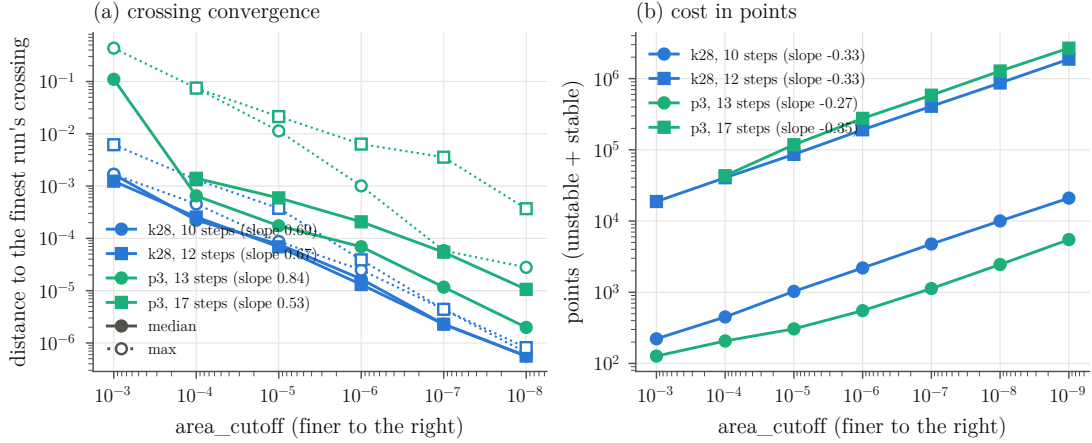


Figure 10: E4 convergence. (a) Distance of each finest-run crossing to the nearest crossing of the coarser run: steady power-law convergence; p3 at 17 steps is the slowest. (b) Cost in points: slopes -0.332 and -0.272, close to the predicted $-1/3$.

6 Physical invariants (E5)

Claim. On every frozen case the numerics respect what the dynamics requires: fixed points to machine precision, monotone cdists, manifolds mapped onto themselves to within the refinement tolerance, and lobe areas preserved.

Evidence. Table 3 summarises the cases. The fixed-point residual is at most 8.9×10^{-16} and the eigenvector residual at most 1.8×10^{-15} . Along the manifolds, 77 328 cdist steps (and 317 502 along bridges) contain 0 decreases and 0 ties. No sampled image lands beyond the tail of its image branch (0 such images). The largest invariance error, 1.1×10^{-3} , belongs to k10, one of the two cases (with inversion) grown at the default 10^{-4} (Figure 11). The 70 registered iterate links agree with the map to a median 6.4×10^{-6} (maximum 1.5×10^{-3} , again k10), so the error tracks each case's cutoff (Figure 12(a)). The 57 lobe/image-lobe pairs have $|A'/A - 1| \leq 5.0 \times 10^{-3}$ with 0 sign flips (Figure 12(b)). The k28 lobes measure ± 7.004 , and their images agree to 5.3×10^{-6} in absolute terms. The p3 lobes are tiny (median 1.1×10^{-4}), so their larger *relative* error corresponds to an absolute error of at most 5.4×10^{-7} , a few times the cutoff.

Interpretation. The area check is the strongest of these results. It compares polygons built from

Table 3: E5 per-case invariants (maxima over orbit points and branches). “arr. image” counts the arrangement regions whose `image_of` was found/missing.

case	outcome	$ f^p(x^*) - x^* $	$ \lambda_u \lambda_s - 1 $	eigvec res.	inv. max	lobe $ A'/A - 1 $	cdist dec./ties (man.)	cdist dec./ties (bridges)	arr. image found/missing
k10	ok	0	1.2×10^{-11}	1.8×10^{-15}	1.1×10^{-3}	8.1×10^{-5}	0/0	0/0	0/7
k28 unblasted	ok	8.9×10^{-16}	0	5.6×10^{-17}	1.1×10^{-5}	9.4×10^{-8}	0/0	0/0	0/3
k28 one blast	ok	8.9×10^{-16}	0	5.6×10^{-17}	1.1×10^{-5}	7.6×10^{-7}	0/0	0/0	1/4
k28 two blasts	ok	8.9×10^{-16}	0	5.6×10^{-17}	1.1×10^{-5}	7.6×10^{-7}	0/0	0/0	1/8
p3	ok	0	4.4×10^{-16}	2.2×10^{-16}	2.4×10^{-5}	5.0×10^{-3}	0/0	0/0	22/3
nested	ok	4.4×10^{-16}	4.4×10^{-16}	2.3×10^{-16}	2.4×10^{-5}	5.0×10^{-3}	0/0	0/0	23/10
inversion	ok	0	0	1.4×10^{-16}	8.6×10^{-4}	7.6×10^{-6}	0/0	0/0	0/4

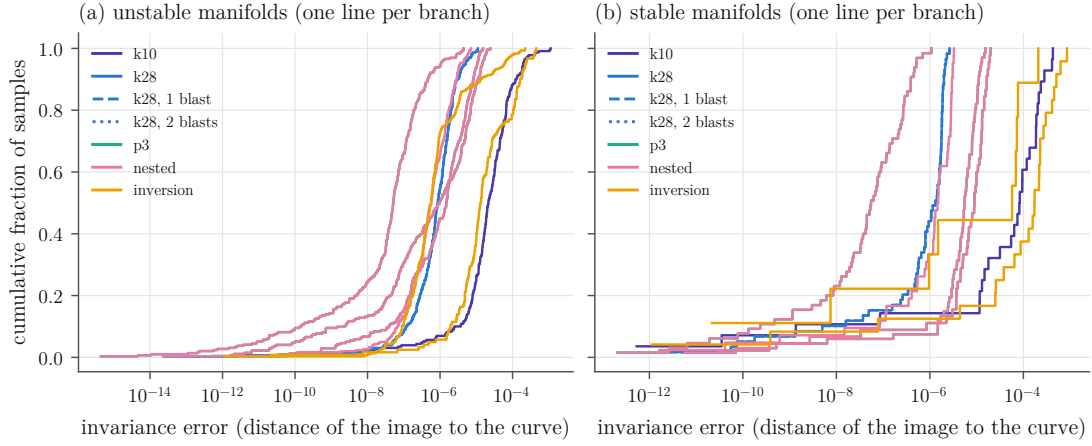


Figure 11: E5 invariance error, one cumulative distribution per branch. The curves to the right are k10 and the inversion saddle, the two cases grown at the default cutoff 10^{-4} ; the 10^{-7} cases sit well to the left.

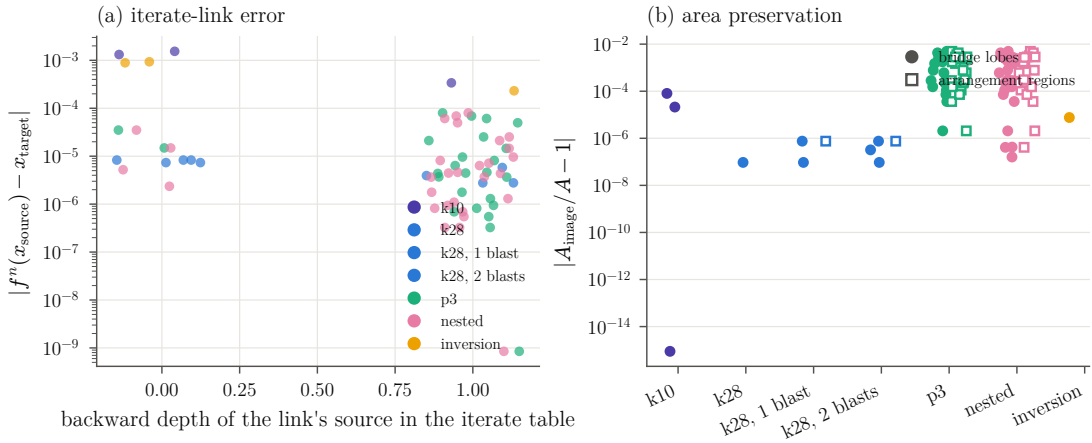


Figure 12: E5. (a) Error of each registered iterate link: the links of the default-cutoff cases (k10, inversion) sit highest, those of the 10^{-7} cases lower. (b) Relative area change of each lobe under one map step (filled) and of each arrangement region with a found image (open). All are $\leq 5.0 \times 10^{-3}$; the largest values belong to the tiny p3 lobes. (The legend overlaps a few points; it hides no outlier.)

independently refined arcs, so a systematic drift would show up as a ratio that grows with depth, and none does.

Caveat. (i) $||\lambda_u \lambda_s| - 1|$ is *weak* evidence: the eigenvalues come from `np.linalg.eig` of the analytic Jacobian, whose determinant is exactly 1, so the defect is 4.4×10^{-16} by construction. The exception confirms the point: k10 passes no analytic Jacobian, its stored Jacobian deviates from the analytic one by 1.4×10^{-12} , and its defect is 1.2×10^{-11} . (ii) The arrangement’s `image_of` found an image for 47 regions and none for 39. It found them for most of p3 and nested, for one region of each blasted k28 case, and for none of k10, unblasted k28 or inversion (Table 3). The data does not say whether this is a gap in `image_of` or an expected consequence of image crossings that are not registered. I list it as an open question (Section 14), not as a bug.

7 Correctness of crossing detection (E9)

Claim. For transverse crossings the R-tree path finds exactly the crossings of the polygons, no more and no less.

Evidence. Over 26 configurations (Table 4, Figure 13), an exact orientation-test brute force found 154 proper $u \times s$ crossings. The registry matched 154, with 0 missed (none needed the near-parallel, zigzag or collision explanations) and 0 extra.

Table 4: E9: crossings against brute force, summed per case. *Some configurations were too large for the $u \times u$ check.

case	configs	brute	matched	near-par.	zigzag	collision	missed unexpl.	extra touch	extra unexpl.	$u \times u$
k10	4	26	26	0	0	0	0	0	0	0*
k28	17	53	53	0	0	0	0	0	0	4*
p3	5	75	75	0	0	0	0	0	0	0

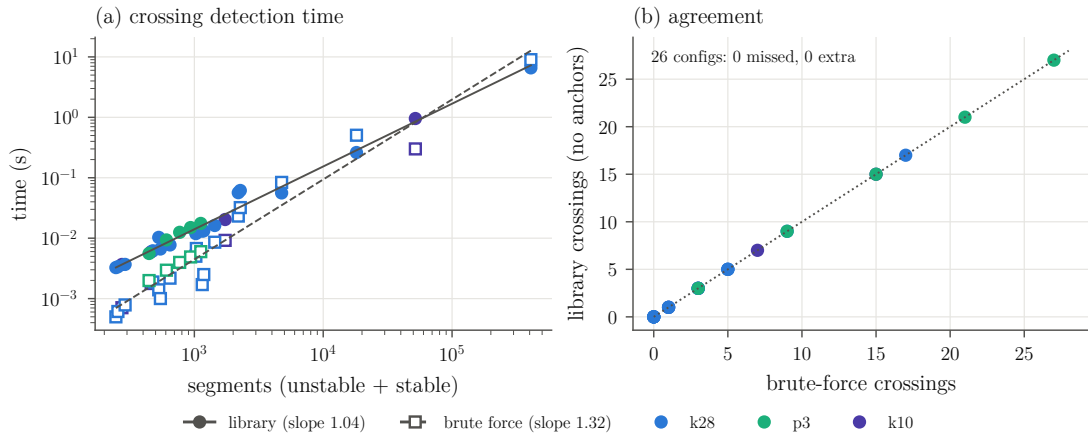


Figure 13: E9. (a) Detection time against segments. The library is linear; the vectorised brute force is steeper but faster on all but the largest configurations. (b) Library against brute-force counts: every point lies on the diagonal.

Interpretation. Speed is not where the R-tree path shines at these sizes: its slope is 1.04 against 1.32 for the brute force, but the brute force was faster in 23 of 26 configurations. On the largest (4.1×10^5 segments) the library took 6.6s and the brute force 9.0s (the largest brute/library ratio was $1.932\times$). This is not a problem: the stable side is short, so the brute force is effectively linear here, and the library time includes registration and bookkeeping. The asymptotic advantage remains with the library.

Caveat. Two gaps. First, the polygons *can* cross themselves: 4 proper $u \times u$ crossings appeared in 2 configurations, all at cutoff 10^{-5} , and 2 large configurations were not checked. A manifold cannot cross itself (CLAUDE.md), so these are discretisation artefacts. The library never tests $u \times u$ pairs, so nobody is told. A cheap self-crossing check would make a good resolution diagnostic. Second, an API trap: `compute_intersections(reset=True)` *replaces* the `IntersectionRegistry` object, so a caller that cached `workbench.intersection_registry` beforehand holds a stale, empty registry. This is documented in the docstring, but easy to trip over (the E6 script carries a comment about it).

8 Robustness across parameters (E6)

Claim. One fixed recipe runs end to end over most of the area-preserving $b = 1$ family. It fails cleanly where it should, but at small k it produces tangles that are numerically fine and topologically empty.

Evidence. Figure 14 and Table 5: 71 of 98 configurations (72.4%) reached the trellis, and 83.3% of the $b = 1$ saddles did. The failure classes are:

- **Small k :** 10 saddles with $k \in [0.30, 0.545]$ grew the unstable side and then raised “Max iterations reached” in `grow_until_turnaround` on the stable side. The smallest completed k is 0.582.
- **Elliptic point:** 6 configurations ($k \leq 2.63$) were rejected as “not a saddle”, by design.
- **Inversion saddle:** 3 runs with $k \geq 10.2$ hit the 4 GB cap during growth, the same blow-up as in Section 9.2.
- **Orientation reversing $b = -1$:** all 8 were rejected (“eigenvalues of disagreeing sign”), by design.
- **Dissipative b :** 15/15 completed. Nothing in the pipeline checks area preservation, so completion here says nothing about correctness.

Table 5: E6: configurations per family by last stage completed.

family	none	fixed point	eigen	seeded	grown unstable	grown	intersected	bridges	trellis pips	total
$b = -1$	8	0	0	0	0	0	0	0	0	8
$b = 0.3$	0	0	0	0	0	0	0	0	5	5
$b = 0.5$	0	0	0	0	0	0	0	0	5	5
$b = 0.9$	0	0	0	0	0	0	0	0	5	5
$b = 1$, other point	6	0	0	3	0	0	0	0	6	15
$b = 1$, saddle	0	0	0	0	10	0	0	0	50	60

Interpretation. “Complete” is not “useful”. Of the 50 completed $b = 1$ saddles, 26 have *no* crossing at all (up to $k = 5.93$). The first crossing appears at $k = 2.50$, and no configuration has more than 3 (Figure 15). With a fixed 8-step recipe the manifolds at small k are simply too short to meet, because λ_u is small. A recipe has to scale its depth with $1/\ln \lambda_u$, as E7’s tangency study does.

Caveat. The “Max iterations” failures are a limit of the stable-side stop condition with this recipe, not necessarily of the map. A larger cap or a different stop condition might rescue them; I did not try either. Note also that the 4 GB cap of E6 is lower than elsewhere.

9 Limits and deep exploration (E7)

Escape and tangency probe failure modes of the numerics; depth and blasts probe how far the pipeline carries a growing tangle, whose topology is expected to change along the way (Section 2).

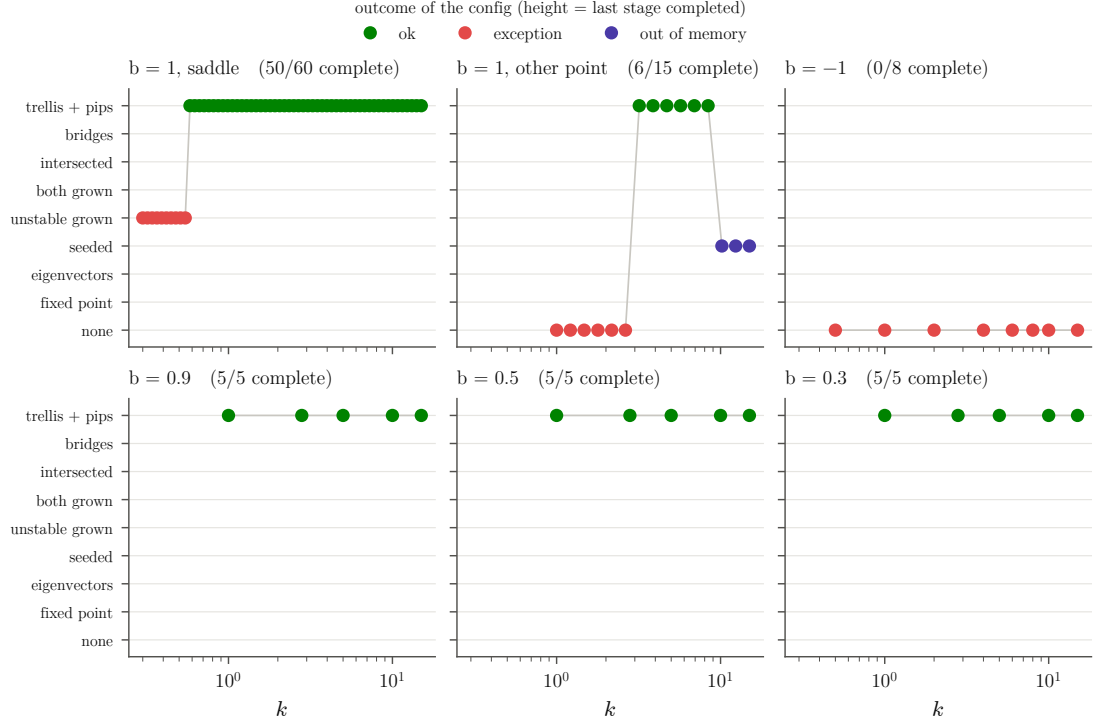


Figure 14: E6. Last stage completed per configuration (height) and outcome (colour). Read the $b = 1$ saddle panel left to right: a block of small- k exceptions, then complete runs. The other point fails as elliptic below $k = 3$ and runs out of memory as the inversion saddle at large k .

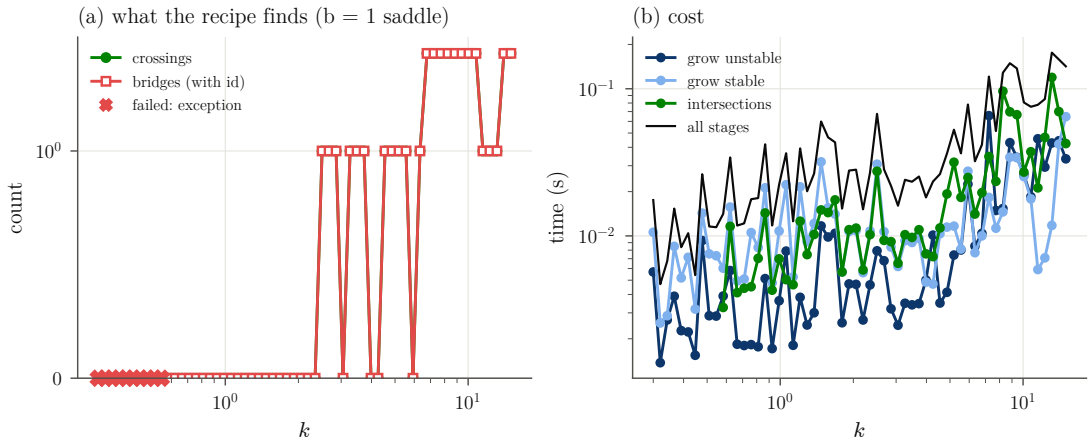


Figure 15: E6 $b = 1$ saddle. (a) Crossings and bridges found by the fixed recipe: none below $k = 2.50$ and intermittent up to $k = 5.93$. (b) Stage times across k : cheap everywhere; cost is not what limits this sweep.

9.1 Escape to infinity

Claim. The documented escape reproduces, and the documented mitigation works.

Evidence. Figure 16: on the $k = 2.1$ period-3 orbit with a fixed step count, the run completed 9 steps holding 9881 points, with $\max|x| = 2.0 \times 10^3$. It was then killed at the 20 GB cap (peak 20 496 MB) after 175 s inside the next step. The same arm grown with a stop condition (arclength 0.5, or $\max|x| > 2.5$) stopped safely at 478 points; arclength 3 stopped at 9881.

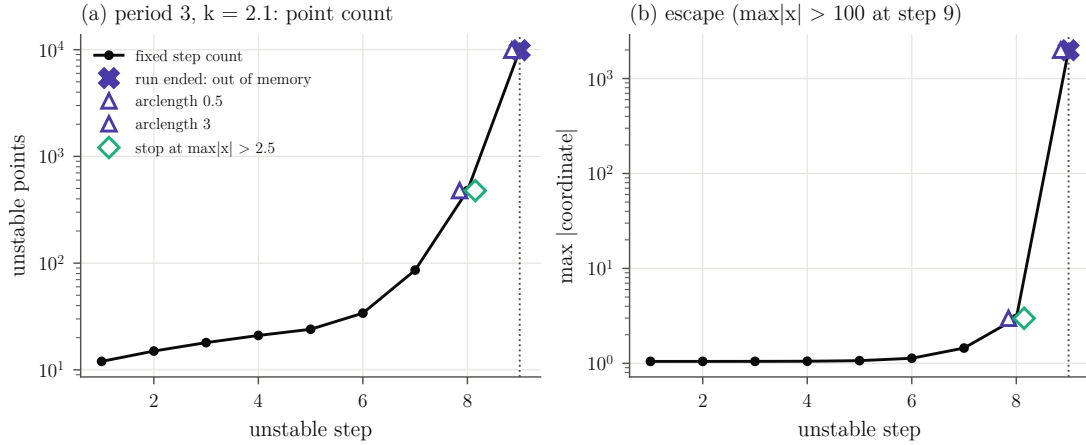


Figure 16: E7 escape ($k = 2.1$, period 3). (a) Points per step: the fixed count dies one step after the curve turns up; the stop conditions (open markers) end before the take-off. (b) The largest coordinate jumps in the last completed step, which is what the $\max|x|$ stop condition catches.

Interpretation. `grow_n_times` has no notion of budget. A single step can go from harmless to tens of gigabytes, so stop-condition growth should be the recommended interface.

9.2 Depth

Claim. p3 is numerically sound through 17 unstable steps, and each extra step uncovers more of the tangle. From 15 steps on, the symbolic layer flags its result as provisional. The inversion saddle hits two genuine errors: a topology invariant assertion when shallow and memory exhaustion when deep.

Evidence. Figure 17: p3 completes the numerics through 17 steps (584 747 points); 18 steps hit the 20 GB cap. The topology stack completes at every depth that finishes the numerics. The tangle it finds grows with depth: the bridge-class count is constant through 14 steps and then rises with every extra step. From 15 steps on, unresolved and virtual classes appear (`is_reliable=False`): some classes cannot be resolved yet (an image not registered, or a dual-graph walk that does not complete) and need more growth or blasting. It does not mean the numerics are wrong. The inversion saddle completes the numerics only up to 6 steps. At 7 and 8 steps it is killed at the cap after reaching 49 417 points at step 6. So the documented “7 steps does not finish” is a memory blow-up, not a hang. At 4 and 5 steps the numerics succeed, but the topology stack trips its own invariant: “crossing 0 at stable cdist 0.0 ... is owned by 0 partition elements”. This is a genuine error (an `AssertionError`), unlike the provisional flag at 6 steps, where the stack completes with a virtual class.

Caveat. The inversion assertion lives in the topology layer and is recorded here because the stress test reached it. Whether the shallow inversion trellis is too thin to partition or the assertion is too strict is for the topology owner to decide. Depth itself is not a fault: a deeper trellis has more crossings and classes by design.

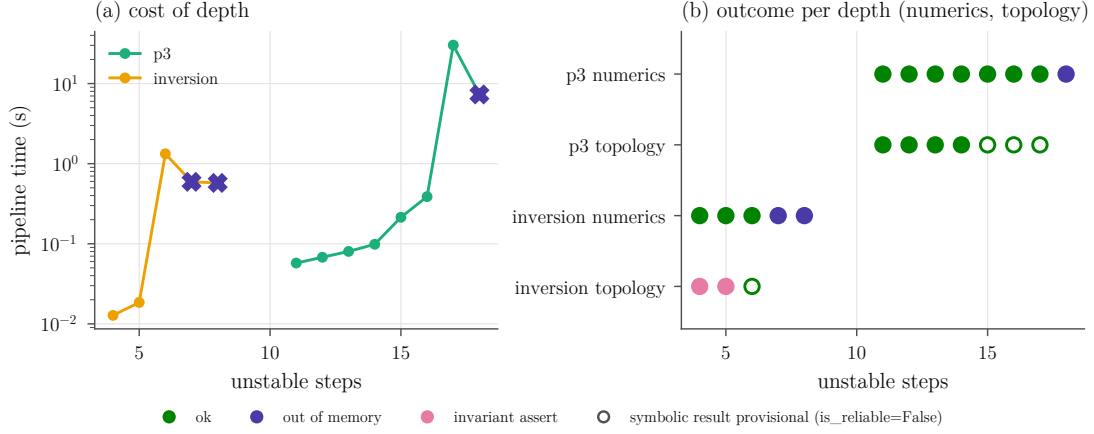


Figure 17: E7 depth. (a) Pipeline time against unstable steps; p3 climbs steeply at the deepest steps. (b) Outcome by layer. The numerics fail only by memory, and the inversion topology raises an assertion at shallow depth. Open circles are topology runs that completed with a provisional symbolic result (`is_reliable=False`: some classes still need more growth). They are not failures.

9.3 Blasts

Evidence. Figure 18: k28 and p3 ran 10 and 10 blasts and nested ran 8, all without numerical error. k28’s crossings grow from 4 to 712, and its last blast took 5.6s. Each blast uncovers new structure, which is the purpose of blasting. After the last blast, k28 has 32 bridge classes, 7 of them still unresolved and 2 virtual, so the symbolic layer marks its result provisional. p3 has 31 classes (1 unresolved, 1 virtual). Nested has 13 classes, all resolved. The topology stack completed in all three cases. No exception, assertion or memory failure occurred.

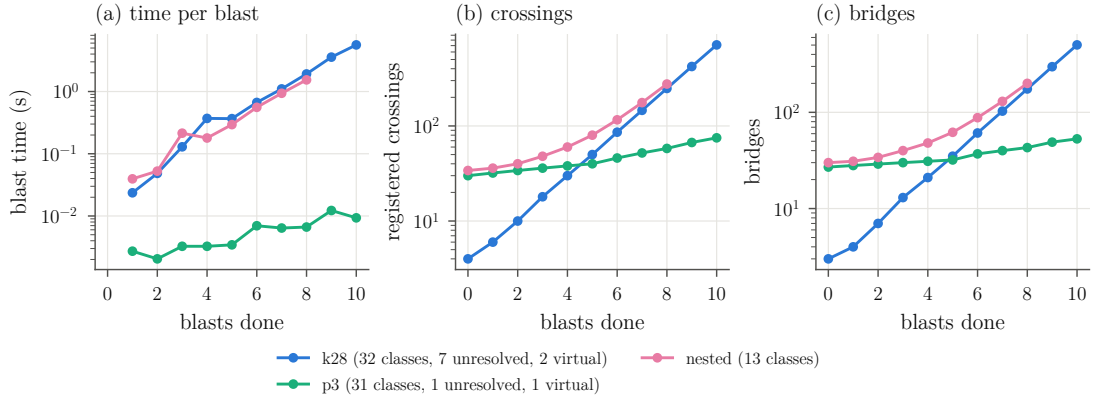


Figure 18: E7 blasts. (a) Time per blast grows geometrically for k28 and nested and stays at milliseconds for p3. (b,c) Crossings and bridges grow geometrically. No numerical error within 8–10 blasts; the growing counts are the new tangle structure each blast reveals.

Interpretation. Blasting is numerically sound to the depth tested and does what it is for: it reveals new crossings, bridges and classes. Unresolved and virtual classes after many blasts are the symbolic layer saying that some classes cannot be resolved yet (E11 records why, Section 10.5). They are open questions for more growth or blasting, not errors.

Caveat. The topology was probed only on the final state, so this experiment does not show how the classes and words changed blast by blast (see Section 10).

9.4 Near tangency: artefact crossings

Claim. This is the most important limitation found. As $k \rightarrow -1$ (the saddle-centre bifurcation) the true separatrix splitting becomes exponentially small. The numerics then register polygon-on-polygon crossings as if they were transverse, their number at fixed k and fixed manifold length is set by the resolution, and no warning fires.

Evidence. Figure 19: 72 configurations, with $k+1$ log-spaced from 10^{-4} to 2.5 and three cutoffs. At $k+1 = 10^{-4}$ the registered crossings are 25, 153 and 499 for cutoffs 10^{-7} , 10^{-10} and 10^{-12} . The peaks over k are 149, 1607 and 4401. From $k+1 \geq 0.16$ on, all three cutoffs agree (e.g. 7 crossings at $k+1 = 1$). At the finest cutoff and $k+1 \leq 0.025$, the smallest $|\sin|$ per configuration lies between 2.0×10^{-6} and 4.9×10^{-5} (median of the medians 4.8×10^{-4}). The heuristic splitting $e^{-\pi^2/\ln \lambda}$ over the same range is 4.6×10^{-31} to 1.9×10^{-8} . Across all of this, warnings of any kind: 0. The smallest angle anywhere, 1.3×10^{-7} , sits at the transition, not near $k = -1$.

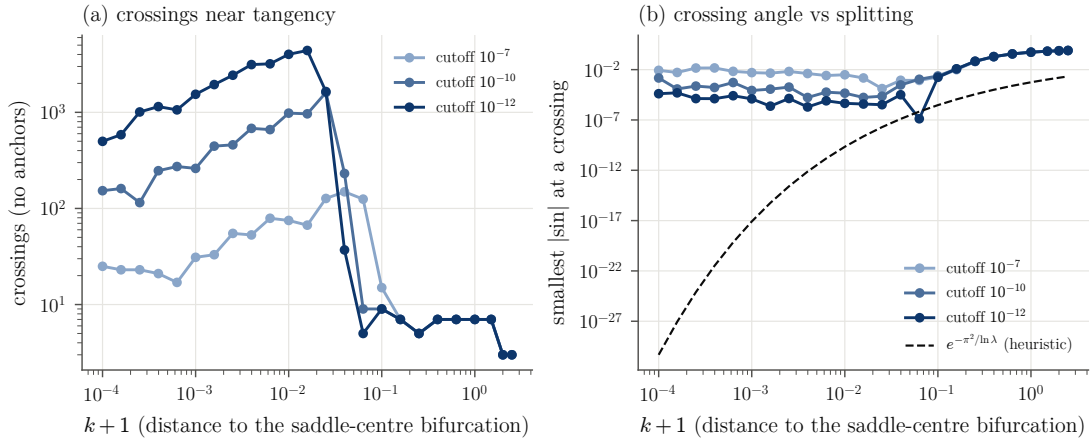


Figure 19: E7 tangency. (a) Crossings against $k+1$: below $k+1 \approx 0.16$ the three cutoffs disagree strongly. That is the signature of artefacts, since a true crossing cannot depend on the polygon’s resolution. (b) Smallest crossing angle against the heuristic splitting (dashed): the registered angles are many decades larger than the true splitting, i.e. they measure polygon noise.

Interpretation. When the true separation of the manifolds is far below the polygon’s own geometric error (Section 5), the two polygons interleave at random. Each interleaving is a clean, well-conditioned segment crossing (median $|\sin|$ 4.8×10^{-4}), far from the library’s relative near-parallel threshold of 10^{-12} , so it is registered as transverse. The crossing tests are correct about the *polygons*; the polygons are not accurate enough to resolve the *manifolds*.

Caveat. The splitting reference is a heuristic order of magnitude, not a computed splitting. Some of the near-tangent crossings may be real; the claim is that their *count* is not trustworthy. A guard that compares each crossing’s angle and position with the local invariance error would flag this regime.

10 Deep blasting: how the topology evolves (E11)

Why this experiment. A blast iterates the bridges inside a resonance zone, which registers crossings that the original growth did not reach. Its purpose is to change the topology: new crossings, bridges and holes, split elements, new bridge classes, new letters, new words and new transition matrices. In this section topology change is the quantity being measured, not a fault (Section 2). E7 blasted three zones and probed the topology once, after the last blast (Section 9.3). E11 blasts five zones up to 20 times and snapshots the whole topology stack (trellis, partitions, minimal trellis, dual graph, symbolic dynamics) after *every* blast. It asks five questions: how the symbolic description evolves, what each blast costs, which parameters change what is uncovered,

how the library’s provisional flags behave as the tangle grows, and whether any genuine error appears.

10.1 Design of the sweep

Cases. k28 and p3 as in E7; k32, the $k = 3.2$ saddle built by the k28 recipe, as a second single saddle; nested_outer, the period-1 zone of the nested map; and nested_inner, its period-3 zone, blasted after the outer zone has been blasted twice (the `build_nested` recipe). Every case is built at `area_cutoff` 10^{-7} , so all cells of one case start from the same tangle; the swept `area_cutoff` is the refinement threshold used *while blasting*.

Grid. The *reference cell* of a case is cutoff 10^{-7} , the recipe’s `min_separation` (10^{-5} , or 10^{-4} for the nested cases) and one iteration per blast. Around it one parameter moves at a time: `min_separation` $\in \{\text{none}, 10^{-6}, 10^{-5}, 10^{-4}, 10^{-3}\}$ (plus 10^{-2} for k28 and k32), `area_cutoff` $\in \{10^{-5}, 10^{-6}, 10^{-8}\}$, `num_iterations` = 2, and two repeats of the reference cell for k28, p3 and nested_inner. That makes 53 configurations, each in a fresh process (900s timeout, 12 GB RSS cap, 8 workers), which took 24 min of wall-clock. The full cutoff $\times$ separation product was dropped for time (Section 15).

Stopping. A configuration ends after 20 blasts (22 configs) or when its own guard declines the next blast: the time budget (timeout minus 120s, 8 configs) or the memory budget ($0.9 \times$ the RSS cap, predicted from the last blast’s growth, 23 configs). These budget stops are the experiment’s resource guard. They keep the last snapshot intact and are not failures; all 53/53 configurations ended ok at the harness level. Topology errors raised inside them are recorded per blast (Section 10.6).

What is recorded. After each blast: the numerics counts (crossings, bridges, points, time, RSS), the two law checks (same-stability crossings; decreasing cdist along a new bridge), a lobe-area probe, the topology counts (classes, active, inert, holes, elements, unresolved, virtual, ambiguous, `is_reliable`) and the topology outcome. A topology exception or `AssertionError` is recorded for that blast and the blasting goes on; such a snapshot has no counts, and the next one is compared with the last snapshot that completed.

Canonical words. The session alphabet never reuses a letter, so a class whose elements a blast has renumbered returns under a new letter even when its dynamics did not change. To tell a relabelling from new symbolic structure, E11 also records *canonical words*: the unrefined words with every letter renamed by its class’s position in the class table (active classes $A_0, A_1, \dots$, inert ones $I_0, I_1, \dots$). A snapshot is *new structure* when its canonical words differ from the previous completed snapshot, a *relabelling* when change flags are set but the canonical words are equal, and *no change* otherwise.

10.2 How the topology evolves

Claim. Blasting keeps uncovering new symbolic structure. On k28 and the period-3 zones it arrives in bursts; k32 changes its words at every completed blast. Nearly every change is a change of the words themselves, not a relabelling.

Evidence. Table 6 and Figure 20. Over all configurations, 490 post-blast snapshots changed the canonical words, only 3 were pure relabellings, and 256 changed nothing.

- *k28* goes from 3 to 57 classes (52 active, 5 inert) and from 4 to 7 792 crossings. Its largest step is at blast 8, from 4 to 20 classes. The first new words appear at blast 1.
- *k32* changes its canonical words at 8 completed blasts and reaches 37 classes at blast 8 (its last completed snapshot), after a step from 15 to 37 at blast 8.

- *p3* and *nested_inner* do not change at all until their first new words (*p3*: blast 6; *nested_inner*: blast 6). There the classes jump from 9 to 23 and from 13 to 27, and then grow by exactly 2 classes per blast to 51 and 55. From the burst on, every blast changes the words (15 and 15 new-structure blasts).
- *nested_outer* rises from 12 to 13 classes at blast 2, then falls from 13 to 11 at blast 10, while its crossings grow from 34 to 9170.

Table 6: E11 per case at the reference cell. “first new words” is the first blast whose canonical words changed; “new-word blasts” counts them; “prov.” marks a provisional symbolic result (*is_reliable=False*). The last two columns cover every configuration of the case.

case	blasts (stop)	crossings	classes	first new words	new-word blasts	unres./virt./amb. at end	topology error from blast	configs w/ error	law viol.
k28	15 (memory budget)	4→7792	3→57	1	4	12 / 1 / 3 (prov.)	9 (zero-width iterated element)	11/12	0
k32	14 (memory budget)	4→6757	3→37 (blast 8)	1	8	2 / 1 / 0 (prov.)	9 (zero-width iterated element)	8/10	0
p3	20 (all blasts)	30→198	9→51	6	15	1 / 1 / 0 (prov.)	none	2/11	0
nested_outer	15 (memory budget)	34→9170	12→11	1	3	0 / 0 / 0	none	0/9	0
nested_inner	20 (all blasts)	40→208	13→55	6	15	1 / 1 / 0 (prov.)	none	2/11	0

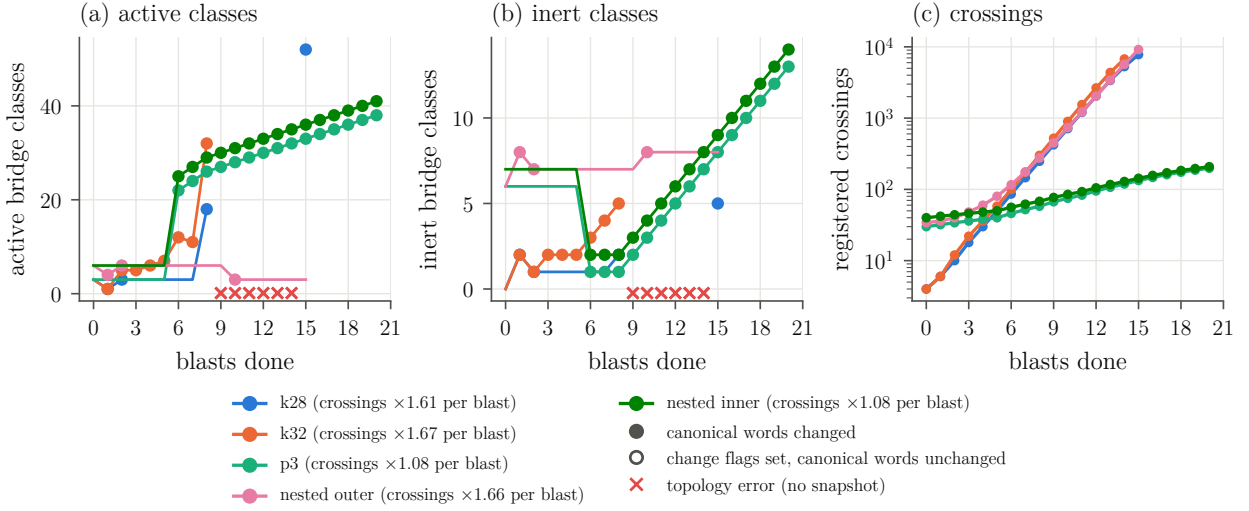


Figure 20: E11 reference cells. (a) Active and (b) inert bridge classes, and (c) registered crossings (log scale), against blasts done. Filled markers: the canonical words changed at that blast (new structure). Red $\times$: the topology raised at that blast, so there is no snapshot. The class counts move in bursts for k28 (blast 8) and for p3 and nested_inner (blast 6); k32’s words change at every completed blast, with its largest class step at blast 8; the crossings grow geometrically for the period-1 zones and slowly for the period-3 ones.

Interpretation. This is blasting doing its job. Each burst is a blast after which the class table is rebuilt over a stable manifold with new holes and cuts, so the class count is recomputed, not appended to; this is also why nested_outer can *lose* classes. Why the bursts fall at those particular blasts (presumably the first blast whose images reach a new part of the stable manifold) I have not checked. The period-3 zones show a regular pattern after their burst, a fixed number of new classes per blast. The period-1 zones have no such pattern, and their crossings grow geometrically rather than slowly.

Caveat. Class counts at a given blast depend on the blasting cutoff (Section 10.4), so these are counts at cutoff 10^{-7} . Snapshots that raised leave gaps: k28 and k32 have none between blasts 9–14, which is why k32’s last count is at blast 8 and k28’s jump to 57 is not resolved blast by blast.

10.3 Cost of blasting: the practical depth limit

Claim. For the large zones (k28, k32, nested_outer) every blast costs a roughly constant factor more than the one before. At cutoff 10^{-7} memory sets how deep one can blast; at coarser cutoffs

the time budget binds first. The period-3 zones cost almost nothing.

Evidence. Figure 21. Per blast, as geometric means over the last five blasts, k28 multiplies its crossings by 1.61, its bridge points by 1.65 and its blast time by 1.69; k32 by 1.67, 1.70 and 1.75; nested_outer by 1.66, 1.68 and 1.77. Their last blasts took 74, 72 and 76 s and peaked at 9.0, 7.6 and 7.5 GB, and the memory budget stopped them after 15, 14 and 15 blasts. Of the 19 large-zone cells at cutoff 10^{-7} with one iteration per blast, 17 were stopped by the memory budget and 2 by the time budget at the same blast count, so which guard binds first there depends on machine load. At the coarser blasting cutoffs 10^{-5} and 10^{-6} the time budget stopped all 6/6 large-zone cells, after 16–17 blasts at 3.1–8.5 GB. p3 and nested_inner grow their crossings by only 1.08 and 1.08 per blast, finish all 20 blasts at a mean of 0.011 and 0.013 s per blast, and peak at 0.42 and 0.47 GB.

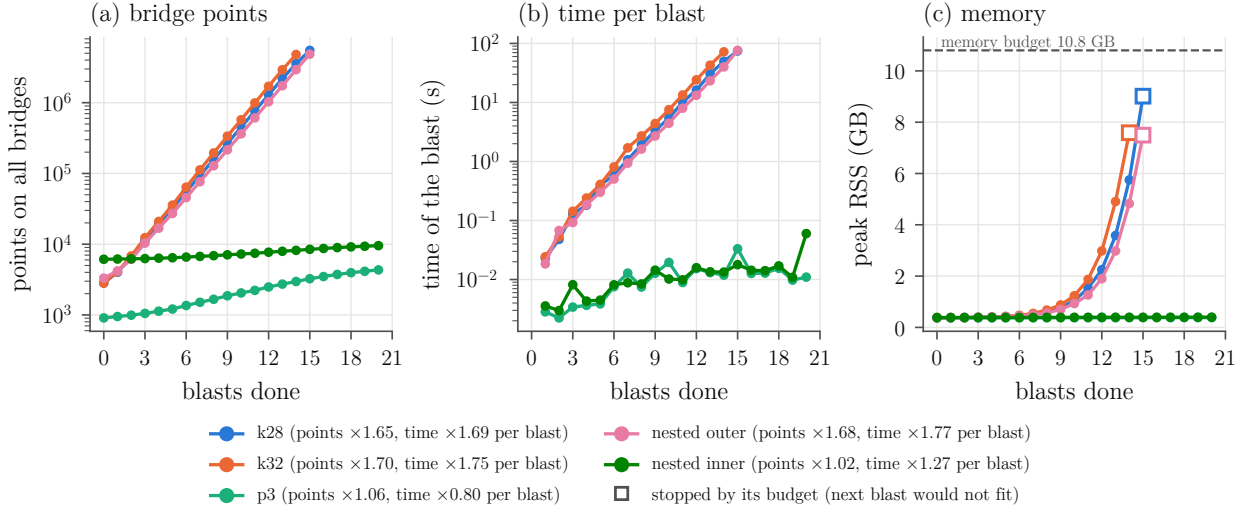


Figure 21: E11 cost at the reference cells. (a) Points on all bridges and (b) time per blast (log scales): straight lines for the period-1 zones, rising only slowly for the period-3 zones (p3’s bridge points go from 908 to 4332). (c) Peak RSS against the memory budget (dashed). Squares mark configurations that their own budget stopped because the next blast would not fit. That is the experiment’s guard, not a failure.

Interpretation. Each blast iterates every interior bridge, and the bridges of a large zone are stretched by the saddle at each step, so points, crossings and time all grow by about the same factor. Because each further blast needs that factor more memory, more hardware buys only a blast or two. This is the same geometric memory growth as unbudgeted growth (Section 9.1); here the harness’s budget stopped it before any OOM. The same cure applies: predict the next blast’s cost from the last factor, as the harness does here, and refuse or warn above a budget. In the period-3 zones the bridge points grow only by 1.06 (p3) and 1.02 (nested_inner) per blast, so deep blasting is cheap.

Caveat. The growth factors are fitted on five blasts. p3’s time factor (0.80) is below one because its blasts take milliseconds and the timing is noise.

10.4 Dependence on parameters, and determinism

Claim. `min_separation` has no effect at any value tested, and the runs are deterministic. The blasting `area_cutoff` changes what the blasts uncover. Two iterations per blast give the same canonical words as two single blasts wherever both completed, in every case but one nested_outer pair.

Evidence. Figures 22 to 24. *Separation and repeats*: every separation cell matches its reference exactly at every blast, in crossings, bridge points, topology outcome and the exact topology hash: 5/5 (k28), 5/5 (k32), 4/4 (p3), 4/4 (nested_outer) and 4/4 (nested_inner). So do the repeats

(2/2, 2/2, 2/2). *Cutoff*: words are compared only at blasts where both snapshots completed; a blast where one cell raised and the other did not is an outcome difference, counted apart. In 12 of the 15 cutoff cells the canonical words leave the reference before the crossing counts do. At 10^{-5} p3's words differ from blast 3 and its crossings from blast 6; k28's from blasts 7 and 11 (its topology outcome first differs at blast 9, where the reference raises). The three exceptions: k28 at 10^{-6} (crossings from blast 10, words from blast 15), k28 at 10^{-8} (crossings from blast 9, words never) and nested_outer at 10^{-8} (crossings from blast 12, words never). After 20 blasts, p3 holds 50, 41, 51 and 51 classes at cutoffs 10^{-5} , 10^{-6} , 10^{-7} and 10^{-8} . *Two iterations*: for $n \geq 1$ (blast 0 is the shared pre-blast build), blast n with two iterations has the same canonical words as single blast $2n$ in 4/4 (k28), 4/4 (k32), 10/10 (p3) and 10/10 (nested_inner) of the pairs where both snapshots completed, and 6/7 for nested_outer (they differ at $n = 5$). The crossing counts agree only up to $n = 4$ (k28), 4 (k32), 3 (nested_outer) and 5 (nested_inner); for p3 they agree up to $n = 10$, every pair compared.

Interpretation. In these zones `min_separation`, even at 10^{-2} , evidently never removes a blast child, so it is not a knob that matters here, and the whole pipeline (blast, repin, topology) is reproducible to the hash. The cutoff does matter: blasted bridges are refined at that cutoff, so a different cutoff can move a crossing from one element to another (new words, same count) before it adds or removes one. As in E4 and the tangency study (Sections 5 and 9.4), a count that depends on the cutoff at a *fixed* blast reflects resolution, while growth with blasts at a fixed cutoff is discovery. On the two-iteration side, the numbers of crossings drift apart after a few blasts, probably because the two routes refine and repin at different moments, but the symbolic words agree wherever both completed, except in one nested_outer pair. Along this route the words are more robust than the raw counts.

Caveat. No cutoff is a ground truth here: the finer 10^{-8} runs stopped earlier, so the cutoff spread at deep blasts is not settled. The nested_outer two-iteration pair whose words differ has not been examined.

10.5 Provisional symbolic results as the tangle grows

Claim. The library's provisional flags come and go as blasting proceeds. A blast can resolve them, and a burst of new classes opens new ones. They are open questions that call for more growth or blasting, not errors.

Evidence. Figure 25. k28, k32 and nested_outer start provisional (an unresolved class before any blast), and the first blast clears it. The result turns provisional again at blast 8 (k28), 6 (k32), 6 (p3) and 6 (nested_inner): at or before each case's class burst. At the end, k28 has 12 unresolved, 1 virtual and 3 ambiguous classes; k32 2, 1 and 0; p3 1, 1 and 0; nested_inner 1, 1 and 0. nested_outer ends with 0 of each and a reliable result. In p3 and nested_inner the counts stay flat from the burst to the last blast.

Interpretation. The library records why each class is unresolved, and the reasons differ before and after the bursts. Before any blast, the 3 unresolved classes of the reference cells are all “a landing is a singleton element and no member has a registered image chain; grow or blast”, and the first blast registers those images, which is exactly the library's “grow or blast until the images are registered”. After the bursts the reason is mostly different: of the unresolved entries in the reference snapshots after blast 0, 44 are “dual-graph walk unreachable” and only 2 the singleton reason (both in k28's last snapshot, next to 10 unreachable walks). In p3 and nested_inner the one unresolved class is an unreachable walk at every blast after the burst. So the burst-era flags are mostly not missing images but walks the dual graph cannot complete. That is still a provisional result for the topology owner (an open question that more growth, or a further rule, may settle), not a numerics fault.

Caveat. The flags are counted per snapshot; I did not check whether the unresolved class is the same class from blast to blast.

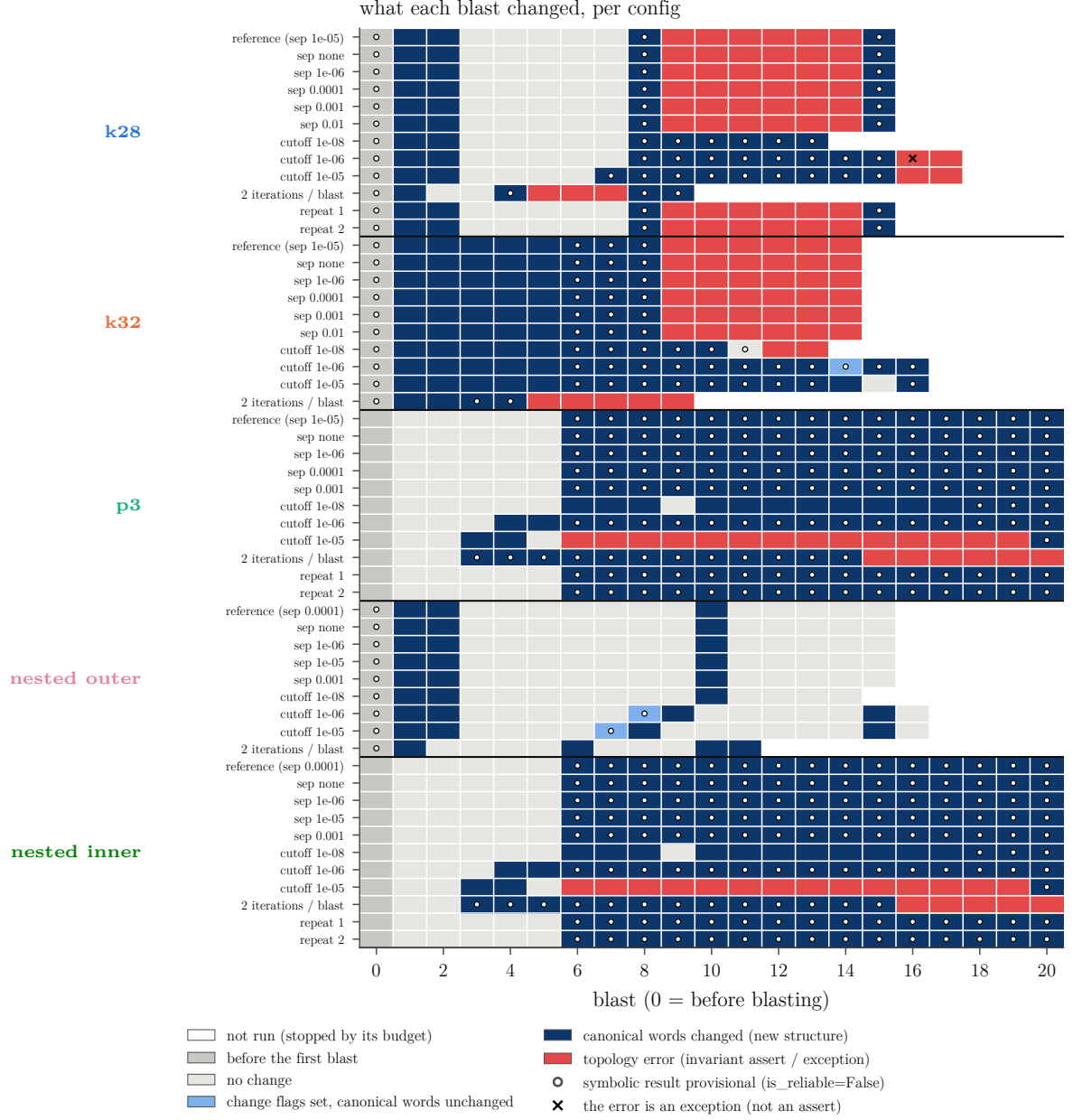


Figure 22: E11: what each blast changed, one row per configuration, grouped by case. Dark blue: new canonical words (new structure). Mid-blue: change flags set but canonical words unchanged (relabelling). Grey: no change. Red: the topology raised (genuine error; $\times$ marks the one exception, the rest are assertions). White: not run, because the configuration's budget stopped it. A white dot marks a provisional symbolic result (`is_reliable=False`). The separation and repeat rows equal their reference row; the cutoff rows do not.

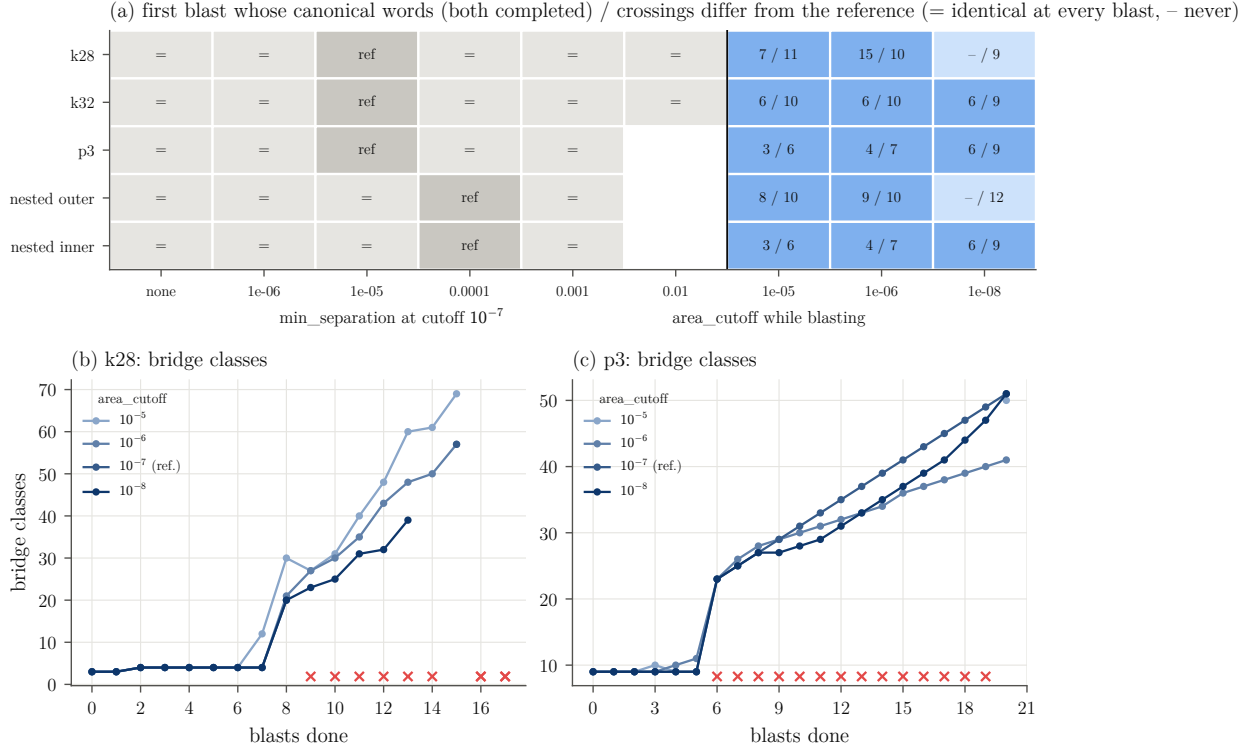


Figure 23: E11 parameters. (a) First blast at which the canonical words / the crossing counts differ from the reference cell, words compared only where both snapshots completed (“=”: identical at every blast; “–”: never). (b,c) Bridge classes against blasts for each blasting cutoff, k28 and p3; red x: snapshots where the topology raised (any cutoff).

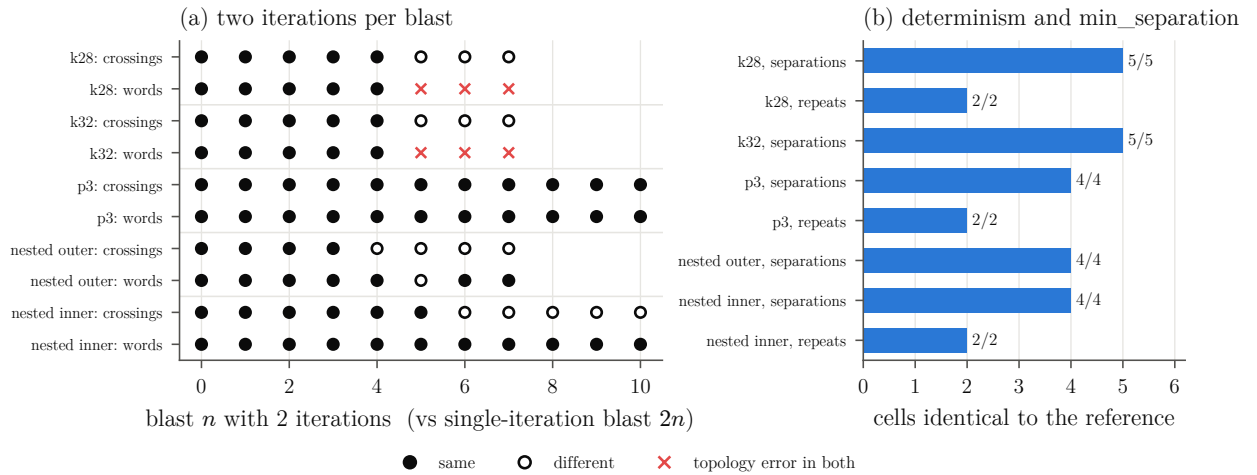


Figure 24: E11 agreement. (a) Blast n with two iterations per blast against single-iteration blast $2n$: crossing counts and canonical words. (b) Separation and repeat cells identical to the reference at every blast.

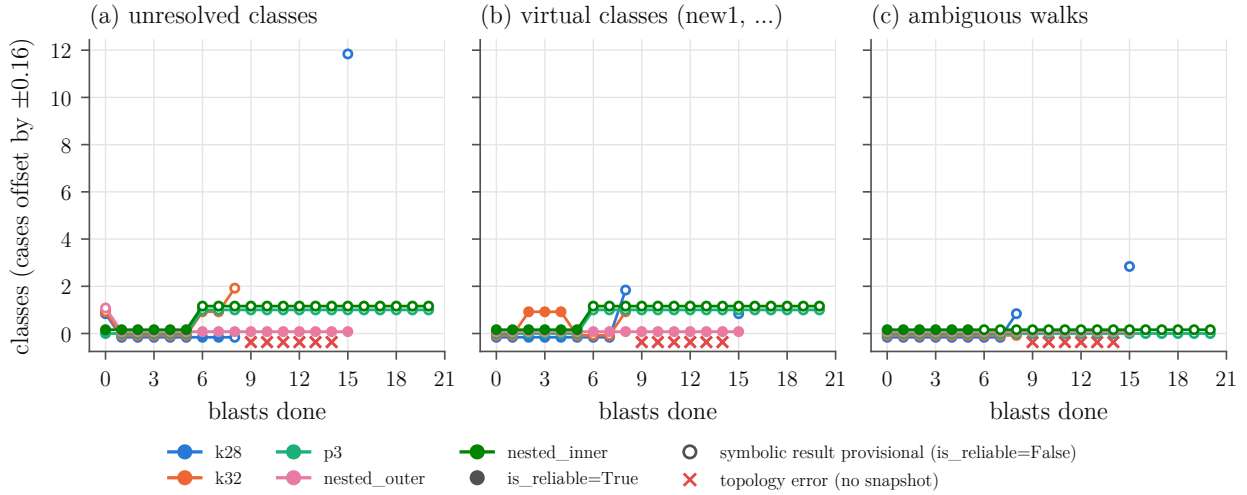


Figure 25: E11 provisional flags at the reference cells: (a) unresolved classes, (b) virtual classes (`new1`, ...), (c) ambiguous walks against blasts. Open markers: provisional symbolic result (`is_reliable=False`); red $\times$: no snapshot (topology error). Cases are offset vertically by ± 0.16 so that they do not overlap.

10.6 Genuine errors

Claim. No same-stability crossing and no decreasing `cdist` appeared at any blast. The area check is inconclusive in the two-iteration cells (see below). The topology layer raised in 137 of 939 snapshots, in 23 of 53 configurations, with 4 distinct errors. These are genuine errors for the topology owner to triage. The blasting itself never failed.

Evidence. Over 886 blasts there were 0 same-stability crossings and 0 decreasing `cdist` steps (0 crossing/`cdist` law violations). The lobe-area probe compared 5 276 lobe pairs; the median of the per-snapshot median $|A'/A - 1|$ is 1.2×10^{-5} (per-pair ratios are not stored), and the reference cells peak at 0.0514. In the two-iteration cells (nested inner 2 iterations / blast, p3 2 iterations / blast) $|A'/A - 1|$ reaches 2.29 with 4 sign flips. The probe compares polygons, which a fold can make self-intersecting, so this is either a self-intersecting probe polygon or a real area defect; I have not decided which, and the area check stays open there. Table 7 lists the topology errors (136 assertions, 1 exception):

1. **Zero-width iterated element “not inside its parent”** (`PartitionFamily.from_minimal`, assertion): 94 snapshots in 17 configurations, blasts 5–14. It is raised at blasts 9–14 in every k28 and k32 cell at cutoff 10^{-7} (all separations and repeats), earlier in blast count (though later in iterations) with two iterations per blast, and in k32 at 10^{-8} . On k28 the snapshot at blast 15 completes again, and k28 at the other cutoffs never raises it.
2. **Holes disagree on bridge_side** (`check_holes_share_bridge_side`, assertion): 3 snapshots in 2 configurations (k28 at cutoffs 10^{-5} and 10^{-6}), blasts 16–17.
3. **Several outer faces** (`DualGraph`, a `ValueError` “component ... faces of positive traversal area”): 1 snapshot, k28 at cutoff 10^{-6} , blast 16.
4. **Bridge approaches its two crossings from opposite sides** (`check_bridge_rows_consistent`, assertion): 39 snapshots in 4 configurations, blasts 6–20: p3 and nested_inner at cutoff 10^{-5} from blast 6 on (the reference cell’s burst; these cells’ own words had changed earlier), and with two iterations per blast near the end of the run.

Interpretation. All four errors live in the topology layer, appear only after several blasts, and none comes with a crossing or `cdist` violation, so they are not caused by the numerics producing impossible crossings. They depend on the blasting cutoff, but that alone does not show they are geometric. E7’s k28 run, built by the same recipe, reached 712 crossings after 10 blasts and its

Table 7: E11 topology errors, per error and case. These are genuine errors; the blasting continued after each, and no crossing or cdist law was violated. “snapshots” counts the blasts at which the error was raised.

error	raised in	kind	case	configs	blasts	snapshots
zero-width iterated element	<code>PartitionFamily.from_minimal</code>	assert	k28	cutoff 10^{-7} , every separation, 2 iterations, repeat 1, repeat 2	5–14	51
			k32	cutoff 10^{-7} , every separation, cutoff 10^{-8} , 2 iterations	5–14	43
several outer faces	<code>DualGraph (arrangement component)</code>	exception	k28	cutoff 10^{-6}	16	1
holes disagree on bridge side	<code>check_holes_share_bridge_side</code>	assert	k28	cutoff 10^{-6} , cutoff 10^{-5}	16–17	3
bridge rows disagree	<code>check_bridge_rows_consistent</code>	assert	p3	cutoff 10^{-5} , 2 iterations	6–20	20
			nested_inner	cutoff 10^{-5} , 2 iterations	6–20	19

topology completed (32 classes, provisional). The E11 k28 reference at the same blast also has 712 crossings, yet it raises the zero-width assertion, as do all k28 cells at cutoff 10^{-7} . So the zero-width error may depend on the E11 snapshot path (a topology stack after every blast, session state left by an assertion mid-stack, or the extra `bridge_classes/minimal_trellis/iterated_partition` calls) rather than on the trellis; it has to be reproduced on a fresh session at that blast before it is attributed to geometry. My working hypotheses, to be checked by the topology owner: (1) an image bridge’s endpoint lands exactly on an existing element boundary, giving an element $[x, x]$, and the parent containment test does not accept a degenerate element at a boundary; (2) the propagated-hole side rule, fixed on 2026-10-02 for the period-3 case, has another corner case on deeply blasted trellises; (3) the arrangement’s outer-face detection meets a component with near-degenerate faces; (4) the row check trips where two crossing signs come out equal, which the row rule reads as a bridge crossing its stable branch. At a coarse cutoff in a folded region that may be a resolution effect. In the two-iteration cells the lobe-area sign flips fall exactly on the first blasts at which this error is raised, which points at the geometry of those bridges rather than at the row rule.

Caveat. Each error’s traceback is stored only at its first occurrence, and a snapshot that raised has no counts. Whether an assertion is a bug or an over-strict invariant is not decided here.

What to investigate next.

- Reproduce the zero-width element at k28’s first failing blast from the stored configuration, on a fresh session that runs the topology stack only once (E7’s k28 run completes at the same crossing count); decide whether degenerate iterated elements are legal (as singletons) and make the containment test closed at boundaries.
- Re-run the bridge-row and hole-side cells with the crossing signs and the offending bridge dumped, to separate a resolution effect from a rule defect.
- Make blasting memory-aware in the library, the way the harness is.
- Follow p3’s persistent unresolved class: grow the stable manifold instead of blasting again, and see whether it resolves.

11 Fixed-point solver (E8)

Claim. The solver is fast and accurate when it finds the orbit you asked for. It finds the full set of short orbits of the $k = 10$ horseshoe. It does *not* check the minimal period, so it regularly hands back a lower-period orbit as if it were period p .

Evidence. 9600 solves (Figure 26, Table 11 in the appendix): 88% converged and 12% did not. 49.7% of all solves gave a saddle of the requested minimal period. The maximum orbit residual is 5.8×10^{-11} , and the median solve takes 0.67 ms (99th percentile 7.2 ms, Figure 28). At $k = 10$ the distinct orbits found equal the full 2-shift count for 6/6 periods (23 of 23 orbits, Figure 27(a)).

Interpretation. The headline wrong-period rate, 34.8% of converged solves, mixes two different things. 1198 solves (14.2%) asked for period 2 at $k < 3$. There is no period-2 orbit there: none

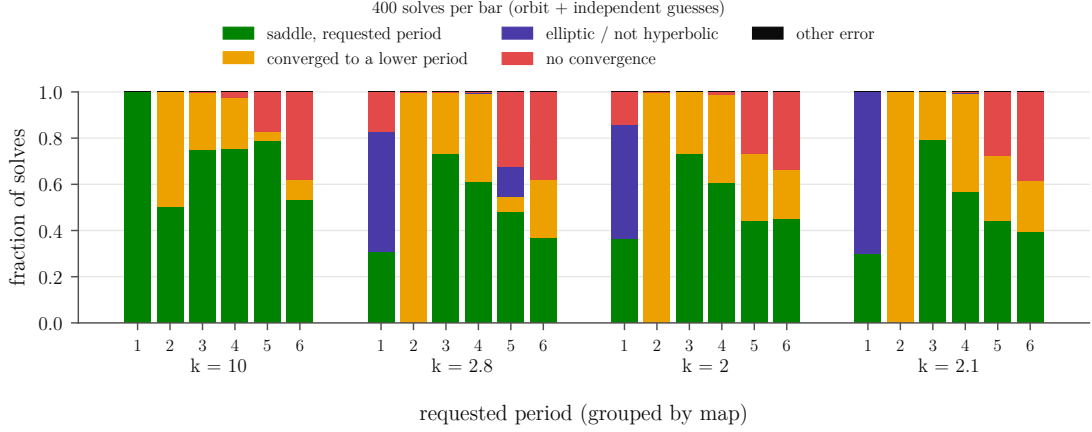


Figure 26: E8 outcomes per map and requested period. Orange (“converged to a lower period”) is the trap; the all-orange period-2 bars at $k < 3$ are maps with no period-2 orbit at all.

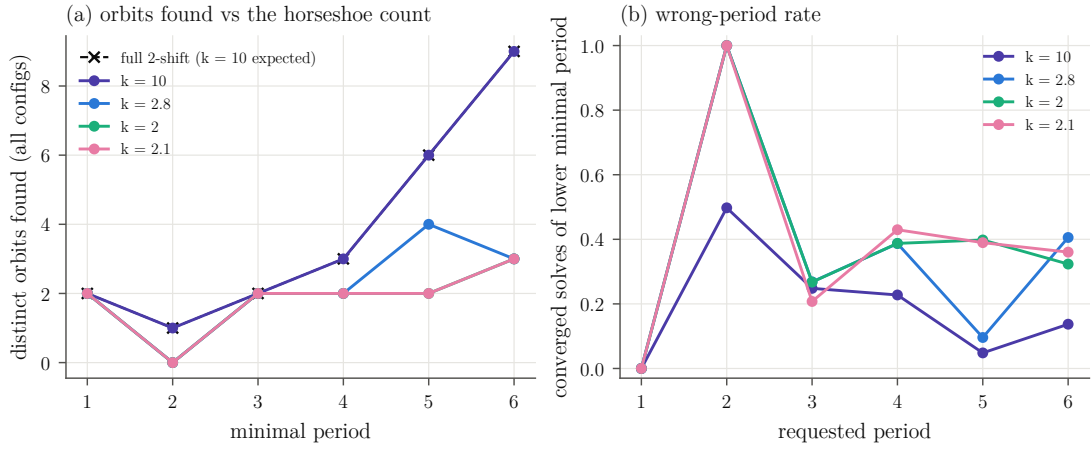


Figure 27: E8. (a) Distinct orbits found: at $k = 10$ the curve lies exactly on the 2-shift count. (b) Fraction of converged solves with a lower minimal period, per requested period.

was found from any seed, consistent with its birth in the period doubling of the elliptic point at $k = 3$. So falling back to a fixed point is the only thing a shooting solver can do. The other 1744 solves (20.6%) are genuine slides: the requested period exists, and the solver found a fixed point or a divisor-period orbit instead. At $k = 10$, where every period exists, the rate is 20.1%. The real problem is downstream: 1424 lower-period solves passed `construct_fixed_point`, which built a saddle with the *requested* period (e.g. a period-1 point labelled period 2 with $k_{\text{value}} = 2$). The lower-period solves that were rejected were rejected for being elliptic, not for their period.

Caveat. Seeds are uniform in a box sized to each map. Basins elsewhere are not sampled, so the orbit counts at $k < 3$ are lower bounds.

12 Reproducibility (E10)

Claim. Runs are deterministic on a given device, and CPU and GPU agree to floating-point rounding.

Evidence. Table 8 and Figure 29: across 5 fresh-process repeats per device, CPU–CPU is identical and GPU–GPU is identical (sha256 of every point array). CPU–GPU differs in 5 of 5 hashes, and still differs in 3 of 5 hashes after rounding to 10^{-12} . The crossing coordinates agree to 2.2×10^{-16} . The CV of the summed stage time is 1.82% (CPU) and 1.21% (GPU); per stage it is at most 5.41% and 4.56%. In this case the GPU run is slower, at $0.85\times$ of the CPU speed (0.112s against

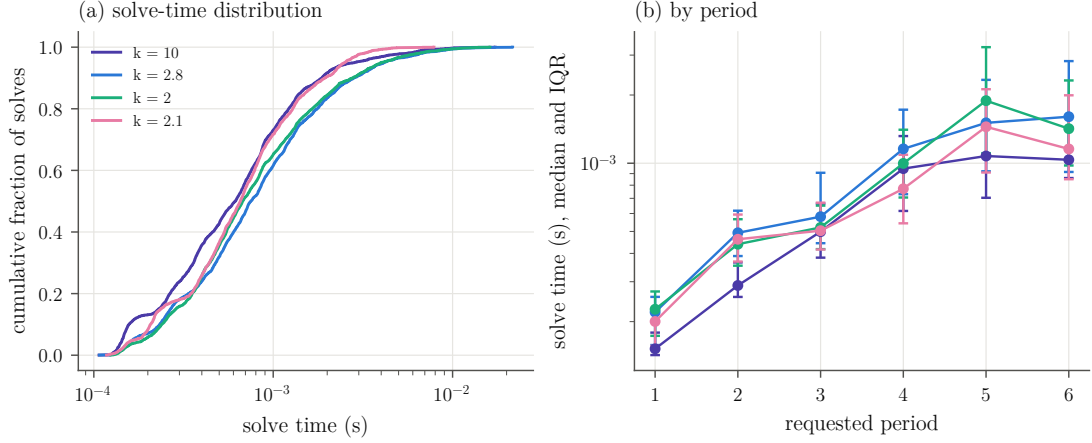


Figure 28: E8 solve time. (a) Distribution: sub-millisecond for most solves. (b) Median and IQR by period: cost grows mildly with the period.

0.132 s).

Table 8: E10: do the sha256 hashes agree across repeats (exact bytes, and rounded to 10^{-12})?

array	exact CPU-CPU	exact GPU-GPU	exact CPU-GPU	10^{-12} CPU-CPU	10^{-12} GPU-GPU	10^{-12} CPU-GPU
unstable	yes	yes	no	yes	yes	no
stable	yes	yes	no	yes	yes	no
crossing	yes	yes	no	yes	yes	yes
cdist	yes	yes	no	yes	yes	yes
bridge	yes	yes	no	yes	yes	no

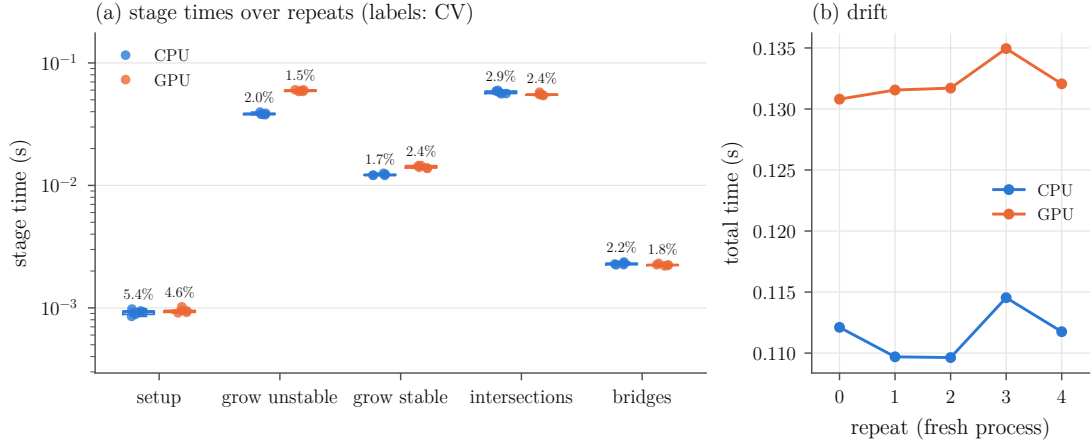


Figure 29: E10. (a) Stage times over five repeats, labelled with their CV: the spread is a few percent. (b) Total time per repeat: no drift, and the GPU is consistently slower on this mid-size case.

Interpretation. The CPU–GPU differences are rounding differences in the map that scale with the coordinate magnitude. In the lockstep comparison (Figure 30) they first appear at step 8. At step 11 of k28 they reach 3.6×10^{-11} in absolute terms but only 1.5×10^{-15} relative to the largest coordinate. At step 12, with coordinates near 5.5×10^8 , they flip refinement decisions: the CPU holds 4348557 points and the GPU 4348603. The crossings, which live near the saddle, are unaffected.

Caveat. Five repeats bound the CV loosely. A timing difference below $\sim 5\%$ between two configurations should not be read as real.

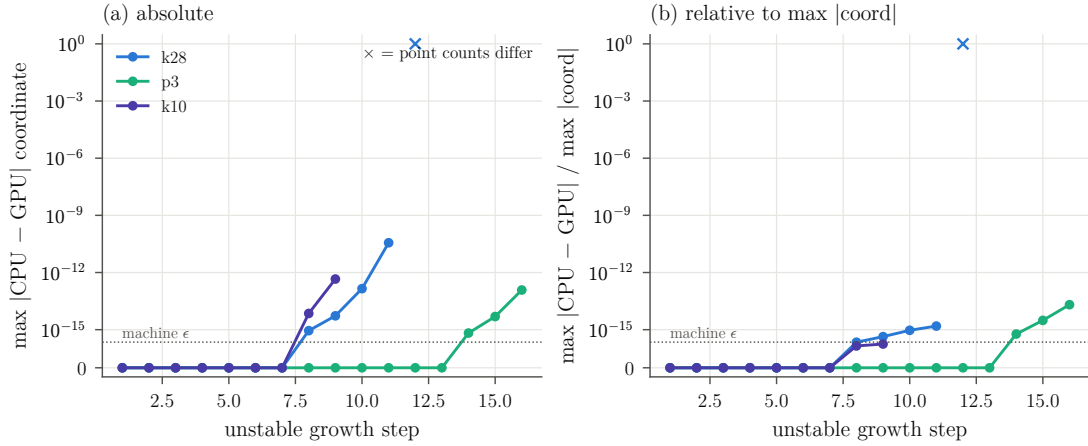


Figure 30: E2 lockstep CPU/GPU deviation of the unstable manifolds. (a) Absolute: it grows once the arm escapes. (b) Relative to the largest coordinate: it stays at machine ϵ . The cross marks the step where the point counts differ.

13 Failure taxonomy

Table 9 collects every failure class the campaign hit. “By design” means the library refuses the input on purpose; “limitation” means correct behaviour within an unstated envelope; “bug” means behaviour the library’s own contract rules out. Topology changing under blasting or deeper growth is not a failure and does not appear here. Neither do the symbolic layer’s provisional flags (Section 2); both are listed after the table.

Expected behaviour (not failures). Blasting and deeper growth change the topology: k28’s crossings went from 4 to 712 over 10 blasts, and p3’s bridge classes rise with every step past 14 (Sections 9.2 and 9.3). In E11, 490 snapshots brought new canonical words, and nested_outer’s classes even fell from 13 to 11 (Section 10.2). That is the tangle being uncovered. The memory and time budget stops of E11 are the experiment’s own guard, not failures.

Provisional symbolic results (open, not errors). The library’s `is_reliable` flag is False for p3 from 15 steps, for the inversion saddle at 6 steps, and after the last blast on k28 (7 unresolved, 2 virtual classes) and p3 (1 unresolved, 1 virtual). In E11 the flags come and go: the first blast clears them on k28, k32 and nested_outer, and they return at the class burst on p3 and nested_inner and at or before it on k28 and k32; nested_outer’s never returns (Section 10.5). Before blasting the unresolved classes lack a registered image; after the bursts they are mostly dual-graph walks that cannot complete. Both call for more growth or a further rule. The arrangement’s `image_of` misses (39 of the regions tried, Section 6) are an open question of the same kind, not a failure.

14 Recommendations

Defaults.

- `area_cutoff`: 10^{-7} . It is converged in every case tested, and its cost over 10^{-4} is the point ratio $450 \rightarrow 4757$ on k28 (Section 5).
- Growth: stop-condition growth (`grow_until_*`) as the documented default. `grow_n_times` only with a budget.
- GPU: off. The measured ceiling is $1.008\times$ (Section 4).

Guards to add (cheap, numerics layer).

Table 9: Failure taxonomy.

failure class	trigger	symptom	kind	suggested fix
Escape OOM	fixed step count past the tangle (Sections 3 and 9.1)	unbudgeted growth: one step goes from MB to the RSS cap; OOM kill	genuine error (OOM)	point/memory budget in <code>grow_n_times</code> ; stop-condition growth by default
Inversion blow-up	inversion saddle at 7 and 8 unstable steps; $k \geq 10.2$ in E6	unbudgeted growth: OOM, documented as “does not finish”	genuine error (OOM)	same budget guard
Artefact crossings	near tangency, $k + 1 < 0.16$ (Section 9.4)	crossing count set by cutoff; no warning	limitation (silent)	angle/invariance-error guard; resolution warning
Self-crossing polygon	coarse cutoff, e.g. 10^{-5} (Section 7)	u×u crossings undetected	limitation (silent)	cheap u×u diagnostic
Coarse-cutoff crossing errors	default 10^{-4} on p3 (Section 5)	crossings invented and missed	limitation	default 10^{-7}
Wrong period	<code>compute_fixed_point</code> from an arbitrary seed (Section 11)	lower-period orbit accepted as period p	bug	minimal-period check in <code>construct_fixed_point</code>
Max iterations	$k \leq 0.545$, <code>grow_until_turnaround</code> (Section 8)	<code>ValueError</code>	limitation	raise cap or change stop condition at small λ_u
Trivial tangle	fixed recipe at small k (Section 8)	completes with 0 crossings	limitation	scale depth with $1/\ln \lambda_u$
Not a saddle	elliptic point	<code>ValueError</code>	by design	none
Orientation reversing	$b = -1$	<code>ValueError</code>	by design	none (out of scope)
Partition assert	inversion at 4 and 5 steps (Section 9.2)	<code>AssertionError</code> “owned by 0 partition elements”	genuine error; bug or too strict (topology)	topology owner to triage
Zero-width iterated element	k28, k32 at blasts 5–14, cutoff dependent (Section 10.6)	<code>AssertionError</code> “not inside its parent” in <code>PartitionFamily.from_minimal</code>	genuine error (topology)	decide whether $[x, x]$ elements are legal; closed containment test
Hole side	k28 at cutoffs 10^{-5} , 10^{-6} , blasts 16–17	<code>AssertionError</code> “disagree on bridge_side”	genuine error (topology)	dump the hole pair; recheck the propagated-side rule
Bridge rows	p3, nested_inner at cutoff 10^{-5} or two iterations, blasts 6–20	<code>AssertionError</code> “opposite sides” (<code>check_bridge_rows_consistent</code>)	genuine error (topology)	dump the crossing signs; separate resolution effect from rule defect
Several outer faces	k28 at cutoff 10^{-6} , blast 16	<code>ValueError</code> in <code>DualGraph</code>	genuine error (topology)	inspect the arrangement component
Stale registry	caching <code>intersection_registry</code> across <code>compute_intersections</code>	empty/stale object	API trap	return or re-fetch; warn in docs

- **Self-crossing diagnostic:** test non-adjacent $u \times u$ (and $s \times s$) segment pairs through the existing R-tree after growth, and warn with the cutoff that would remove them.
- **Angle/resolution warning:** for each registered crossing, compare $|\sin \theta|$ times the local segment length with the local invariance error, and warn when the crossing cannot be distinguished from polygon noise.
- **Minimal-period check:** in `construct_fixed_point`, reject (or relabel) an orbit whose minimal period is a proper divisor of the requested one.
- **Memory guard:** predict the next step's points from the last growth factor, and refuse or warn above a budget. The E3 harness already does this in a few lines.
- **Memory-aware blasting:** the same guard for `blast_zone`, since a large zone multiplies its points by about 1.65 per blast (Section 10.3). The E11 harness's memory budget is a template.

Guards to add (topology layer, for its owner).

- **Zero-width iterated elements:** either admit a degenerate element $[x, x]$ as a singleton or merge it into a neighbour before the parent-containment assertion in `PartitionFamily.from_minimal` (Section 10.6).
- **Diagnostics on assertion:** have the row and hole-side checks report the offending crossings and their signs, so that a resolution effect can be told from a rule defect.

Open questions for the advisor.

1. Near tangency, should we accept that crossings below a resolution threshold are undecidable with polygons, or invest in a validated method (interval arithmetic, or parameterisation-method manifolds)?
2. Is `image_of` expected to fail for regions whose image crossings are not registered (39 of the regions tried), or is this a gap?
3. Should dissipative maps ($b \neq \pm 1$) be in scope? They currently run without complaint and without validation.
4. Is the inversion partition assertion at shallow depth a real defect or an over-strict invariant?
5. Do the unresolved and virtual classes after many blasts resolve with further growth? In `p3` and `nested_inner` they persist through every blast after the burst (Section 10.5), so further blasting of the same zone does not resolve them; stable growth is untested.
6. Which blasting cutoff should define the symbolic result? In most cutoff cells the words diverge before the crossing counts do (Section 10.4).
7. Is the lobe-area excess in the two-iteration E11 cells ($|A'/A - 1|$ up to 2.29, 4 sign flips) a self-intersecting probe polygon or a real area defect (Section 10.6)?

15 Threats to validity

- **One machine.** All timings are from a single host and one GPU. The GPU conclusion rests on Amdahl's law, which transfers to other hardware; the absolute crossover sizes do not.
- **Hénon only.** Every result is for the Hénon family. Maps with more expensive evaluations would raise f_{map} and could change the GPU verdict.
- **Quick calibration of grids.** The sweep grids and depths were calibrated in a short pilot to fit the budget. Exponents rest on ≤ 3 decades, and some limits (escape and depth OOM, the tangency transition) are only known to the resolution of the grid. The E7 blast study ran one cutoff per case, with the topology probed only after the last blast.

- **E11 grid thinned and budgets.** E11 moved one parameter at a time around a reference cell; the full cutoff $\times$ separation product and repeats of the heavy k32 and nested_outer cells were dropped. Its heavy cells were stopped by their own memory or time budget (23 and 8 configs), so the deep behaviour of the large zones beyond 15 blasts is unknown, and cutoffs are compared at different final depths. Snapshots that raised leave gaps in the series.
- **cProfile overhead.** Profiling inflates Python call costs relative to NumPy kernels, which biases f_{map} low (see the caveat in Section 4).
- **RSS sampling.** The watchdog samples every 50 ms, so peak RSS can be missed by a short spike, and OOM kills happen up to one sample after the cap. Caps differ per experiment (Table 12).
- **Invariance probe design.** The probe maps points sampled inside segments, because a vertex’s image is often exactly a vertex, and measures one-step consistency. It is not a distance to the true manifold.
- **References are finest runs.** Convergence in E4 is measured against 10^{-9} , and E9’s “truth” is a brute force on the *same* polygons. E9 therefore validates the intersection code, not the manifold approximation.

A Tables

Table 10: E4: cost and accuracy per cutoff. “crossing dev.” is the median distance of the finest run’s crossings to this run’s.

case	steps	cutoff	outcome	points	time (s)	inv. p50	inv. p99	crossings	crossing dev.
k28	10	10^{-3}	ok	222	9.9×10^{-3}	3.7×10^{-4}	5.6×10^{-3}	5	1.7×10^{-3}
k28	10	10^{-4}	ok	450	0.0148	1.3×10^{-4}	9.6×10^{-4}	5	2.2×10^{-4}
k28	10	10^{-5}	ok	1 029	0.0274	3.2×10^{-5}	2.2×10^{-4}	5	7.7×10^{-5}
k28	10	10^{-6}	ok	2 194	0.0511	1.0×10^{-5}	4.8×10^{-5}	5	1.7×10^{-5}
k28	10	10^{-7}	ok	4 757	0.107	1.8×10^{-6}	6.9×10^{-6}	5	2.3×10^{-6}
k28	10	10^{-8}	ok	10 028	0.247	3.6×10^{-7}	1.7×10^{-6}	5	5.6×10^{-7}
k28	10	10^{-9}	ok	20 988	0.515	7.9×10^{-8}	4.4×10^{-7}	5	ref.
k28	12	10^{-3}	ok	18 801	0.449	2.3×10^{-4}	5.6×10^{-3}	17	1.2×10^{-3}
k28	12	10^{-4}	ok	40 604	1.01	1.3×10^{-4}	7.3×10^{-4}	17	2.6×10^{-4}
k28	12	10^{-5}	ok	86 407	2.18	3.2×10^{-5}	1.5×10^{-4}	17	6.9×10^{-5}
k28	12	10^{-6}	ok	191 411	5.22	1.0×10^{-5}	3.4×10^{-5}	17	1.3×10^{-5}
k28	12	10^{-7}	ok	408 846	11.6	1.8×10^{-6}	6.9×10^{-6}	17	2.3×10^{-6}
k28	12	10^{-8}	ok	868 883	23.9	3.6×10^{-7}	1.7×10^{-6}	17	5.6×10^{-7}
k28	12	10^{-9}	ok	1 875 771	54.6	7.9×10^{-8}	4.4×10^{-7}	17	ref.
p3	13	10^{-3}	ok	127	0.0195	8.3×10^{-5}	2.7×10^{-3}	14	0.11
p3	13	10^{-4}	ok	207	0.0179	1.1×10^{-4}	1.4×10^{-3}	37	6.5×10^{-4}
p3	13	10^{-5}	ok	306	0.021	1.2×10^{-4}	6.7×10^{-4}	27	1.7×10^{-4}
p3	13	10^{-6}	ok	552	0.0367	3.0×10^{-5}	9.1×10^{-5}	27	6.9×10^{-5}
p3	13	10^{-7}	ok	1 127	0.0462	8.9×10^{-6}	1.9×10^{-5}	27	1.2×10^{-5}
p3	13	10^{-8}	ok	2 451	0.0932	1.7×10^{-6}	4.4×10^{-6}	27	2.0×10^{-6}
p3	13	10^{-9}	ok	5 485	0.175	3.9×10^{-7}	9.0×10^{-7}	27	ref.
p3	17	10^{-3}	oom	n/a	n/a	n/a	n/a	n/a	n/a
p3	17	10^{-4}	ok	43 340	1.09	3.0×10^{-4}	1.4×10^{-3}	249	1.4×10^{-3}
p3	17	10^{-5}	ok	117 917	3.14	1.0×10^{-4}	4.0×10^{-4}	214	6.0×10^{-4}
p3	17	10^{-6}	ok	276 531	7.49	2.2×10^{-5}	9.7×10^{-5}	213	2.1×10^{-4}
p3	17	10^{-7}	ok	585 323	16.5	4.9×10^{-6}	2.0×10^{-5}	203	5.4×10^{-5}
p3	17	10^{-8}	ok	1 278 043	36.7	1.0×10^{-6}	3.9×10^{-6}	203	1.1×10^{-5}
p3	17	10^{-9}	ok	2 677 865	77.7	2.2×10^{-7}	8.7×10^{-7}	203	ref.

Table 11: E8: solver outcomes per map and requested period. “orbits found” counts distinct orbits of exactly that minimal period over all solves (for $k = 10$: found / full 2-shift count).

map	period	converged	saddle	lower period	orbits found	max residual
$k = 10$	1	100%	100%	0	2/2	0
$k = 10$	2	100%	50.2%	49.8%	1/1	5.8×10^{-14}
$k = 10$	3	99.8%	75%	24.8%	2/2	8.2×10^{-13}
$k = 10$	4	97.8%	75.5%	22.8%	3/3	5.8×10^{-11}
$k = 10$	5	83%	79%	4.82%	6/6	4.7×10^{-12}
$k = 10$	6	62%	53.5%	13.7%	9/9	1.8×10^{-11}
$k = 2$	1	85.8%	36.5%	0	2	8.9×10^{-16}
$k = 2$	2	99.8%	0	100%	0	6.2×10^{-15}
$k = 2$	3	100%	73.2%	26.8%	2	1.1×10^{-13}
$k = 2$	4	98.8%	60.5%	38.7%	2	6.6×10^{-13}
$k = 2$	5	73.5%	44.2%	39.8%	2	7.6×10^{-13}
$k = 2$	6	66.5%	45%	32.3%	3	3.5×10^{-12}
$k = 2.1$	1	100%	30%	0	2	1.3×10^{-15}
$k = 2.1$	2	100%	0	100%	0	2.7×10^{-15}
$k = 2.1$	3	100%	79.2%	20.8%	2	2.8×10^{-13}
$k = 2.1$	4	99.5%	56.8%	43%	2	8.3×10^{-13}
$k = 2.1$	5	72.5%	44.2%	39%	2	1.3×10^{-12}
$k = 2.1$	6	61.8%	39.5%	36%	3	1.6×10^{-12}
$k = 2.8$	1	83%	31%	0	2	8.9×10^{-16}
$k = 2.8$	2	99.8%	0	100%	0	1.7×10^{-14}
$k = 2.8$	3	99.8%	73%	26.8%	2	2.2×10^{-13}
$k = 2.8$	4	99.5%	61%	38.7%	2	1.7×10^{-12}
$k = 2.8$	5	67.8%	48.2%	9.59%	4	4.1×10^{-12}
$k = 2.8$	6	62.3%	37%	40.6%	3	1.5×10^{-11}

B Reproduction

From the repository root, with the project’s environment:

```
env/bin/python stress/numerics/run_campaign.py --quick # pilot
env/bin/python stress/numerics/run_campaign.py # full run
env/bin/python stress/numerics/report/make_figures.py
env/bin/python stress/numerics/report/make_numbers.py
env/bin/python stress/numerics/report/make_numbers_extra.py
cd stress/numerics/report && ~/.local/bin/tectonic report.tex
```

Results land in `stress/numerics/results/*.jsonl` (one record per configuration) with a running `progress.log`. A single experiment runs on its own as

```
env/bin/python stress/numerics/eNN_*.py [--quick] [--only I,J] \
  [--rss-cap-gb G] [--timeout S] [--workers N]
```

E11 is not part of `run_campaign.py`; it runs on its own (8 workers, up to 12 GB each), before the report scripts above:

```
env/bin/python stress/numerics/e11_blast_depth.py # full grid
env/bin/python stress/numerics/e11_blast_depth.py --quick # smoke test
```

C Configuration and outcome counts

Table 12: Records and outcomes per experiment in the full run, with each experiment’s default per-configuration timeout and RSS cap (some E7 configs override the timeout). “assert” counts `invariant_assert` outcomes of the numerics; the topology assertions of E7 and E11 are genuine errors recorded inside `ok` records (E11’s are in Table 7).

experiment	configs	ok	exception	assert	oom	timeout	timeout (s)	cap (GB)
e01_map_throughput	8	8	0	0	0	0	300	24
e02_gpu_end_to_end	33	33	0	0	0	0	600	24
e03_scaling	22	19	0	0	3	0	900	24
e04_area_cutoff	28	27	0	0	1	0	900	16
e05_invariants	8	8	0	0	0	0	600	12
e06_parameter_sweep	98	71	24	0	3	0	240	4
e07_breaking	92	88	0	0	4	0	600	20
e08_fixed_point_solver	24	24	0	0	0	0	900	24
e09_intersections	26	26	0	0	0	0	600	16
e10_determinism	10	10	0	0	0	0	300	24
e11_blast_depth	53	53	0	0	0	0	900	12
total	402	367	24	0	11	0		